

Agentic Sales Orchestrator

Account and Contact Programmatic Extension Runbook

Enterprise implementation guide with true line-by-line C# script explanation

Item	Value
Environment	Phoenicarix-CI - https://phoenicarix-ci.crm4.dynamics.com
Solution	ASO.Core / unique name ASOCore
Tables	Account (account) and Contact (contact)
Implementation method	.NET 8 console applications using Microsoft.PowerPlatform.Dataaverse.Client
Version	v1.0
Date	2026-05-23
Audience	Customer IT team, product owners, junior developers, senior consultants, non-technical stakeholders

Executive summary

We programmatically extended the existing Microsoft Dataverse Account and Contact tables in the Phoenicarix-CI environment. The objective was to complete the standard-table part of the ASO Phase 2 data model after Lead and Opportunity had already been extended. The scripts created ASO-specific columns inside the ASO.Core solution, reused global choices where required, created local choices where values are table-specific, skipped columns that already existed, and published the table customizations. In simple terms: we added governed boxes to Account and Contact records so later AI, Customer Insights, SAP-aware context, and communication-governance components have a safe place to store data.

Business objective

Prepare Account and Contact records for the Agentic Sales Orchestrator MVP by adding fields that support account intelligence, consent, communication lifecycle tracking, Customer Insights journey writeback, preference tracking, and future SAP-aware context. The changes do not activate automation or customer communication; they create the data foundation.

Technical objective

Use repeatable .NET 8 console scripts with Microsoft.PowerPlatform.Dataverse.Client to create Dataverse metadata in bulk. The scripts target the existing standard tables `account` and `contact`, associate created columns with solution `ASOCore`, and publish targeted table metadata.

Scope

In scope: Account extension, Contact extension, Dataverse metadata creation, global choice reuse, local choice creation, targeted publishing, validation in ASO.Core, managed/unmanaged backup after completion. Out of scope: form layout changes, Power Automate flows, Customer Insights journey activation, SAP calls, HubSpot integration, security role hardening, data migration, sample records, and production deployment.

Architecture and context

Dataverse remains the ASO operational data layer for seller-facing records. Account stores organization-level intelligence and customer lifecycle state. Contact stores person-level consent, preference, and Customer Insights communication state. Customer Insights remains the future lifecycle communication execution layer; these fields only prepare the data contract. SAP fields are references/context only and do not represent direct SAP integration.

Why programmatic extension

The maker UI is reliable for small changes, but manually creating many fields across Account and Contact is slow and error-prone. Programmatic creation is faster, repeatable, reviewable, idempotent, and solution-aware. The script checks existing columns first, skips duplicates, and uses the ASOCore solution unique name so the created metadata can be exported and backed up correctly.

Prerequisites

Required before execution: Phoenicarix-CI Dataverse environment, ASO.Core solution with unique name ASOCore, publisher prefix aso, global choices already created (`aso\_communicationstate`, `aso\_lifecyclecommunicationstage`, `aso\_journeyparticipationstatus`, `aso\_emailconsentstatus`), .NET 8 on macOS, Microsoft.PowerPlatform.Dataverse.Client package, interactive OAuth access to Dataverse, and a managed/unmanaged ASO.Core backup before schema changes.

Permissions and security roles

The executing user must be allowed to customize Dataverse, create attributes on standard tables, publish customizations, and add components to the ASO.Core solution. In practice this normally requires System Administrator or System Customizer privileges in the target environment. If permissions are incomplete, the script may connect but fail when creating metadata or publishing.

Backup and ALM approach

Before each schema batch, ASO.Core was exported as unmanaged and managed. Unmanaged backup is useful for recovery and continued development in the source environment. Managed backup is useful to preserve deployable output and to test the downstream managed packaging story. The agreed naming pattern is `

Implementation commands used

Account

```
mkdir -p ~/aso-dataverse-tools/account-extension
cd ~/aso-dataverse-tools/account-extension
dotnet new console --framework net8.0
dotnet add package Microsoft.PowerPlatform.Dataverse.Client
# Program.cs was populated with the Account script
dotnet run
```

Contact

```
mkdir -p ~/aso-dataverse-tools/contact-extension
cd ~/aso-dataverse-tools/contact-extension
dotnet new console --framework net8.0
dotnet add package Microsoft.PowerPlatform.Dataverse.Client
# Program.cs was populated with the Contact script
dotnet run
```

Account field inventory

Display name	Schema/logical name	Type	Size / values	Business purpose
SAP Business Partner ID	aso_sapbusinesspartnerid	Text	100	Reference to SAP Business Partner ID at account level.
SAP Customer ID	aso_sapcustomerid	Text	100	Reference to SAP Customer ID at account level.
AI Account Health	aso_aiaccounthealth	Text	100	Stores account health classification as text for MVP flexibility.
AI Growth Summary	aso_aigrowthsummary	Multiline text	4000	Stores AI-generated growth context for the account.
AI Renewal Risk	aso_airenewalrisk	Text	100	Stores renewal risk signal as text because taxonomy may evolve.
AI Last Run On	aso_ailastrunon	Date and time	User local	Timestamp for latest AI/account context refresh.
AI SAP Account Rich Context	aso_aisapaccountrichcontext	Multiline text	4000	Stores governed SAP account context summary.
AI Correlation ID	aso_aicorrelationid	Text	100	Trace key to link Dataverse updates to future orchestration/observability.
Communication State	aso_communicationstate	Global choice	aso_communicationstate	Reusable communication governance state.
Lifecycle Communication Stage	aso_lifecyclecommunicationstage	Global choice	aso_lifecyclecommunicationstage	Reusable lifecycle stage for communication orchestration.

Journey Participation Status	aso_journeyparticipationstatus	Global choice	aso_journeyparticipationstatus	Reusable journey participation status.
Customer Insights Journey ID	aso_customerinsightsjourneyid	Text	100	Latest related Customer Insights journey identifier.
Customer Insights Journey Name	aso_customerinsightsjourneyname	Text	200	Latest related Customer Insights journey name.
Customer Insights Last Entry On	aso_customerinsightslastentryon	Date and time	User local	Timestamp for last journey entry.
Customer Insights Last Interaction On	aso_customerinsightslastinteractionon	Date and time	User local	Timestamp for latest Customer Insights interaction.
Customer Insights Last Interaction Type	aso_customerinsightslastinteractiontype	Local choice	EmailSent, Open, Click, FormSubmit, Reply, Unsubscribe, Bounce, CustomAction	Type of the latest Customer Insights interaction.
Email Consent Status	aso_emailconsentstatus	Global choice	aso_emailconsentstatus	Reusable consent status for communication eligibility.
Compliance Profile Name	aso_complianceprofilename	Text	200	Compliance profile used for Customer Insights communication governance.
Communication Hold Reason	aso_communicationholdreason	Multiline text	4000	Reason communication is blocked, suppressed, or held.

Contact field inventory

Display name	Schema/logical name	Type	Size / values	Business purpose
Communication State	aso_communicationstate	Global choice	aso_communicationstate	Reusable communication governance state for the person/contact.
Lifecycle Communication Stage	aso_lifecyclecommunicationstage	Global choice	aso_lifecyclecommunicationstage	Reusable lifecycle stage for contact communication.
Journey Participation Status	aso_journeyparticipationstatus	Global choice	aso_journeyparticipationstatus	Reusable Customer Insights journey participation status.
Customer Insights Journey ID	aso_customerinsightsjourneyid	Text	100	Latest related Customer Insights journey identifier.
Customer Insights Journey Name	aso_customerinsightsjourneyname	Text	200	Latest related Customer Insights journey name.
Customer Insights Last Entry On	aso_customerinsightslastentryon	Date and time	User local	Timestamp for last journey entry.
Customer Insights Last Interaction On	aso_customerinsightslastinteractionon	Date and time	User local	Timestamp for latest Customer Insights interaction.
Customer Insights Last Interaction Type	aso_customerinsightslastinteractiontype	Local choice	EmailSent, Open, Click, FormSubmit, Reply, Unsubscribe, Bounce, CustomAction	Type of the latest Customer Insights interaction.
Email Consent Status	aso_emailconsentstatus	Global choice	aso_emailconsentstatus	Reusable consent status for communication eligibility.
Compliance Profile Name	aso_complianceprofilename	Text	200	Compliance profile used for Customer Insights communication governance.
Preferred Email Address	aso_preferredemailaddress	Text	200	Preferred email address to support communication preference handling.
Preference Center Last Visited On	aso_preferencecenterlastvisitedon	Date and time	User local	Timestamp when the preference center was last visited.

How the scripts work end to end

1. Define the Dataverse environment URL (<https://phoenicarix-ci.crm4.dynamics.com>).
2. Define the solution unique name (ASOCore).
3. Define the target table logical name (account or contact).
4. Build an OAuth connection string.
5. Create a ServiceClient.
6. Stop immediately if the connection is not ready.
7. Read existing table columns using RetrieveEntityRequest.

8. Build a list of AttributeMetadata objects representing desired ASO fields.
9. Loop through every planned field.
10. Skip the field if it already exists.
11. Create missing fields with CreateAttributeRequest.
12. Attach created fields to ASO.Core using SolutionUniqueName = ASOCore.
13. Print success/failure messages.
14. Publish the target table with PublishXmlRequest.
15. Validate the result in Power Apps.

Validation checklist

Check	How to validate	Expected result
Correct environment	Power Apps top-right environment selector	Phoenicarix-CI
Correct solution	Power Apps -> Solutions	ASO.Core exists and is opened
Account columns	ASO.Core -> Tables -> Account -> Columns -> search aso_	Account ASO columns are visible
Contact columns	ASO.Core -> Tables -> Contact -> Columns -> search aso_	Contact ASO columns are visible
Choice reuse	Open Communication State, Lifecycle Communication Stage, Journey Participation Status, Email Consent Status	They reuse the existing global choices
Script success	Terminal output	Green CREATED messages and final Done message
Publishing	Refresh maker portal after script	Columns are visible and usable
Backups	Solutions export history / downloaded zip files	Managed and unmanaged ASO.Core backups exist

Troubleshooting guide

Symptom	Likely cause	Resolution
Connection failed	Wrong URL, expired login, or insufficient access	Authenticate again and confirm Phoenicarix-CI URL.
Global choice error	Referenced global choice logical name does not exist or differs	Open ASO.Core -> Choices and confirm logical names.
No Main method / entry point	Program.cs was not pasted correctly	Replace Program.cs with the full script and save.
Field already exists	Script was rerun or field was manually created earlier	Usually safe. Validate data type if unsure.
Fields created but not in ASO.Core	Wrong SolutionUniqueName or solution association issue	Add existing columns to ASO.Core and correct script solution name.
Publish fails	Insufficient privileges or metadata lock	Retry after permissions check; publish manually if needed.
German UI shows localized table names	Standard Dataverse tables are localized in UI	Use logical names account and contact in scripts, not display labels.

Risks and mitigations

Risk	Mitigation
Creating schema in the wrong environment	Print and verify DataverseUrl before execution; use PAC auth list and Power Apps environment selector.
Columns outside the intended solution	Use SolutionUniqueName = ASOCore and validate components under ASO.Core.
Wrong schema name	Review Program.cs before running. Schema names are difficult to rename later.
Wrong choice strategy	Reuse global choices only where already defined; use local choices for table-specific interaction type.
Accidental production impact	This is a trial/MVP environment; do not run against production without change control.
Partial execution	Scripts are idempotent and skip existing fields; rerun after fixing the error.

Rollback and recovery approach

Dataverse schema rollback is not the same as data rollback. If something goes wrong, first stop and document the exact terminal output. If the issue is partial creation, fix the script and rerun because existing fields are skipped. If fields were created with wrong names or types and no data depends on them, remove them manually from ASO.Core/table designer after confirming dependencies. If solution state is severely compromised, use the exported managed/unmanaged backups as recovery references. In a production-grade environment, schema changes should be tested in DEV and moved through managed solutions rather than directly scripted in production.

Handover notes for customer IT

- Store the scripts in source control.
- Record the execution date, user, environment URL, and solution unique name.
- Keep both managed and unmanaged backups after each schema milestone.
- Do not activate customer journeys or flows just because fields exist.
- Add form tabs/views later as a separate controlled change.
- Confirm field-level security and editability before production use.
- For CI/CD, replace interactive OAuth with an approved service principal approach.

PART 2 - Full script documentation with line-by-line explanation

The following sections explain every non-blank code line in the exact Account and Contact scripts used for this implementation. Structural braces and statement terminators are included where they affect code structure. The table is intentionally detailed so a junior developer can learn from it and a senior developer can review it.

Account script - line-by-line explanation

Line	Code	Explanation in simple language	Technical explanation	Why it matters	Common mistake / warning
1	using Microsoft.Crm.Sdk.Messages;	Imports Microsoft.Crm.Sdk.Messages so the script can use Dataverse and C# SDK classes.	C# namespaces provide the types used later, such as ServiceClient, CreateAttributeRequest, RetrieveEntityRequest, and metadata classes.	Without the correct imports, the compiler cannot resolve SDK types.	Removing required using statements causes compile errors such as type or namespace not found.
2	using Microsoft.PowerPlatform.Dataverse.Client;	Imports Microsoft.PowerPlatform.Dataverse.Client so the script can use Dataverse and C# SDK classes.	C# namespaces provide the types used later, such as ServiceClient, CreateAttributeRequest, RetrieveEntityRequest, and metadata classes.	Without the correct imports, the compiler cannot resolve SDK types.	Removing required using statements causes compile errors such as type or namespace not found.
3	using Microsoft.Xrm.Sdk;	Imports Microsoft.Xrm.Sdk so the script can use Dataverse and C# SDK classes.	C# namespaces provide the types used later, such as ServiceClient, CreateAttributeRequest, RetrieveEntityRequest, and metadata classes.	Without the correct imports, the compiler cannot resolve SDK types.	Removing required using statements causes compile errors such as type or namespace not found.
4	using Microsoft.Xrm.Sdk.Messages;	Imports Microsoft.Xrm.Sdk.Messages so the script can use Dataverse and C# SDK classes.	C# namespaces provide the types used later, such as ServiceClient, CreateAttributeRequest, RetrieveEntityRequest, and metadata classes.	Without the correct imports, the compiler cannot resolve SDK types.	Removing required using statements causes compile errors such as type or namespace not found.
5	using Microsoft.Xrm.Sdk.Metadata;	Imports Microsoft.Xrm.Sdk.Metadata so the script can use Dataverse and C# SDK classes.	C# namespaces provide the types used later, such as ServiceClient, CreateAttributeRequest, RetrieveEntityRequest, and metadata classes.	Without the correct imports, the compiler cannot resolve SDK types.	Removing required using statements causes compile errors such as type or namespace not found.
7	const string DataverseUrl = "https://phoenixarix-ci.crm4.dynamics.com";	Stores the target Dataverse environment URL: Phoenixarix-CI.	The ServiceClient connection string later reads this constant to decide which Dataverse organization receives metadata changes.	It prevents the script from accidentally targeting the wrong environment when reviewed before execution.	A wrong URL creates schema in the wrong environment or fails authentication.
8	const string SolutionUniqueName = "ASOCore";	Stores the solution unique name ASOCore.	CreateAttributeRequest and CreateEntityRequest can use SolutionUniqueName to associate created components with a solution layer.	This keeps columns inside ASOCore instead of unmanaged/default solution context only.	Using the display name instead of unique name, or a typo, can leave components outside the intended solution.
9	const string EntityLogicalName = "account";	Stores the target Dataverse table logical name: account.	Dataverse SDK calls use logical names, not localized display names. This tells the script which standard table to extend.	It ensures the script extends Account, not another table.	Using a display name like Account or Kontakt instead of account/contact causes SDK failures.
10	const int LanguageCode = 1033;	Sets language code 1033 for labels.	1033 is English. Dataverse labels and descriptions are localized; this selects the label language for DisplayName and Description.	It makes generated labels readable and consistent in the metadata.	If multilingual labels are required, additional localized labels must be added later.
12	var connectionString =	Begins building the Dataverse connection string.	The following lines provide OAuth authentication parameters for ServiceClient.	The script must authenticate before it can read or create Dataverse metadata.	Incomplete connection strings lead to connection failures.
13	\$@"AuthType=OAuth;	Uses OAuth authentication.	OAuth is the authentication type used by ServiceClient for interactive Microsoft identity sign-in.	It allows the maker/admin user to sign in securely from macOS.	Hardcoding passwords is not acceptable; OAuth is safer for this MVP script.
14	Url={DataverseUrl};	Injects the Dataverse URL into the connection string.	The interpolated connection string substitutes DataverseUrl at runtime.	This keeps the environment target centralized in one constant.	Do not replace this with a different environment unless intentionally redeploying.
15	LoginPrompt=Auto;	Allows the login prompt to appear automatically if needed.	ServiceClient can open an interactive browser/device login when no valid token is cached.	This is convenient for local script execution on a Mac.	In CI/CD this should be replaced with non-interactive authentication.
16	ClientId=51f81489-12ee-4a9e-aaae-a2591f45987d;	Provides the public client ID used for interactive Dataverse tooling authentication.	The GUID is a Microsoft tooling client ID commonly used in Dataverse SDK samples for local OAuth login.	It allows ServiceClient to authenticate without creating a custom app registration for the trial MVP.	For production automation, use an approved application registration and secret/certificate strategy.
17	RedirectUri=http://localhost";	Sets localhost as the OAuth redirect URI.	After browser login, the authentication response returns to the local process.	This enables interactive login to complete on the developer machine.	Changing this can break the local login flow.
19	using var service = new ServiceClient(connectionString);	Imports var service = new ServiceClient(connectionString) so the script can use Dataverse and C# SDK classes.	C# namespaces provide the types used later, such as ServiceClient, CreateAttributeRequest, RetrieveEntityRequest, and metadata classes.	Without the correct imports, the compiler cannot resolve SDK types.	Removing required using statements causes compile errors such as type or namespace not found.

21	if (!service.IsReady)	Checks whether the Dataverse connection is ready.	ServiceClient exposes IsReady to confirm authentication and connection initialization succeeded.	It prevents schema changes from running against a failed or partial connection.	Skipping this check can hide authentication errors until later SDK calls fail.
22	{	Opens or closes a C# block or object initializer.	Braces and terminators define scope and statement boundaries in C#.	They keep methods, loops, conditionals, and metadata objects syntactically valid.	Missing or extra braces/semicolons cause compile errors or wrong structure.
23	Console.ForegroundColor = ConsoleColor.Red;	Changes terminal output color to red for errors.	Console.ForegroundColor controls text color in the terminal.	Operators can quickly identify failures.	Always reset color afterward to avoid confusing output.
24	Console.WriteLine("Connection failed:");	Prints a clear connection failure heading.	This message is shown only inside the failed connection branch.	It tells the operator to fix authentication or environment targeting before continuing.	Do not ignore this message; the script stops after it.
25	Console.WriteLine(service.LastError);	Prints the detailed connection error.	ServiceClient.LastError contains the most recent connection or request error message.	It gives actionable troubleshooting information.	The error may contain authentication or permission details useful for IT.
26	Console.ResetColor();	Resets terminal text color back to normal.	Console.ResetColor clears the previous ForegroundColor setting.	It keeps terminal output readable after colored status lines.	If missing, later output may stay red/yellow/green.
27	return;	Stops the script immediately.	return exits the current top-level program flow.	It prevents metadata operations after connection failure.	Removing it may allow the script to continue with an unusable service object.
28	}	Opens or closes a C# block or object initializer.	Braces and terminators define scope and statement boundaries in C#.	They keep methods, loops, conditionals, and metadata objects syntactically valid.	Missing or extra braces/semicolons cause compile errors or wrong structure.
30	Console.WriteLine(\$"Connected to: {DataverseUrl}");	Prints the connected environment URL.	String interpolation injects DataverseUrl into terminal output.	The operator can visually confirm the target environment before changes are made.	If this shows the wrong URL, stop before continuing.
31	Console.WriteLine(\$"Target solution: {SolutionUniqueName}");	Prints the target solution unique name.	This outputs ASOCore from the constant.	It confirms the target solution layer used for the new metadata.	If it is wrong, fields may not appear in ASO.Core.
32	Console.WriteLine(\$"Target table: {EntityLogicalName}");	Prints the target table logical name.	This outputs account or contact from EntityLogicalName.	It confirms whether the script is extending Account or Contact.	Stop if the table is not the intended one.
33	Console.WriteLine();	Writes a blank line to the terminal.	This is a formatting line only.	It makes logs easier to read.	No business or Dataverse effect.
35	var existingAttributes = GetExistingAttributeLogicalNames(service, EntityLogicalName);	Reads the columns that already exist on the target table.	Calls a helper that executes RetrieveEntityRequest and returns current attribute logical names.	This allows the script to skip existing fields safely.	Without this, rerunning the script would likely fail on duplicate schema names.
37	var fields = new List<AttributeMetadata>	Starts the list of columns the script intends to create.	AttributeMetadata is the base class for Dataverse column metadata definitions.	This list is the implementation data contract for the table extension.	Every field added here should be reviewed before execution.
38	{	Opens or closes a C# block or object initializer.	Braces and terminators define scope and statement boundaries in C#.	They keep methods, loops, conditionals, and metadata objects syntactically valid.	Missing or extra braces/semicolons cause compile errors or wrong structure.
39	Text("aso_sapbusinesspartnerid", "SAP Business Partner ID", 100, "SAP Business Partner reference."),	Defines the SAP Business Partner ID column (aso_sapbusinesspartnerid).	Uses StringAttributeMetadata with a maximum length. This is suitable for IDs, names, labels, and references. In this script the field purpose is: Reference to SAP Business Partner ID at account level.	Creates a short text column. It provides a governed place for reference to sap business partner id at account level.	Wrong length or schema names can create hard-to-change metadata.
40	Text("aso_sapcustomerid", "SAP Customer ID", 100, "SAP Customer reference."),	Defines the SAP Customer ID column (aso_sapcustomerid).	Uses StringAttributeMetadata with a maximum length. This is suitable for IDs, names, labels, and references. In this script the field purpose is: Reference to SAP Customer ID at account level.	Creates a short text column. It provides a governed place for reference to sap customer id at account level.	Wrong length or schema names can create hard-to-change metadata.
42	Text("aso_aicounthealth", "AI Account Health", 100, "AI account health. Kept as text in MVP because taxonomy may evolve."),	Defines the AI Account Health column (aso_aicounthealth).	Uses StringAttributeMetadata with a maximum length. This is suitable for IDs, names, labels, and references. In this script the field purpose is: Stores account health classification as text for MVP flexibility.	Creates a short text column. It provides a governed place for stores account health classification as text for mvp flexibility.	Wrong length or schema names can create hard-to-change metadata.
43	Memo("aso_aigrowthsummary", "AI Growth Summary", "AI growth summary."),	Defines the AI Growth Summary column (aso_aigrowthsummary).	Uses MemoAttributeMetadata with MaxLength = 4000 so longer summaries and rationale can be stored. In this script the field purpose is: Stores AI-generated growth context for the account.	Creates a multiline text column. It provides a governed place for stores ai-generated growth context for the account.	Do not use short text for AI summaries or hold reasons because content may be truncated.
44	Text("aso_airenwalrisk", "AI Renewal Risk", 100, "AI renewal risk. Kept as text in MVP because taxonomy may evolve."),	Defines the AI Renewal Risk column (aso_airenwalrisk).	Uses StringAttributeMetadata with a maximum length. This is suitable for IDs, names, labels, and references. In this script the field purpose is: Stores renewal risk signal as text because taxonomy may evolve.	Creates a short text column. It provides a governed place for stores renewal risk signal as text because taxonomy may evolve.	Wrong length or schema names can create hard-to-change metadata.
45	DateTimeField("aso_ailastrunon", "AI Last Run On", "Last AI run timestamp."),	Defines the AI Last Run On column (aso_ailastrunon).	Uses DateTimeAttributeMetadata with DateTimeFormat.DateAndTime and DateTimeBehavior.UserLocal. In this script the field purpose is: Timestamp for latest AI/account context refresh.	Creates a date/time column. It provides a governed place for timestamp for latest ai/account context refresh.	Wrong datetime behavior can confuse users in different time zones.
46	Memo("aso_aisapaccountrichcontext", "AI SAP Account Rich Context", "SAP account rich context."),	Defines the AI SAP Account Rich Context column (aso_aisapaccountrichcontext).	Uses MemoAttributeMetadata with MaxLength = 4000 so longer summaries and rationale can be stored. In this script the field purpose is: Stores governed SAP account context summary.	Creates a multiline text column. It provides a governed place for stores governed sap account context summary.	Do not use short text for AI summaries or hold reasons because content may be truncated.
47	Text("aso_aicorrelationid", "AI Correlation ID", 100, "Last run correlation."),	Defines the AI Correlation ID column (aso_aicorrelationid).	Uses StringAttributeMetadata with a maximum length. This is suitable for IDs, names, labels, and references. In this script the field purpose is: Trace key to link Dataverse updates to future orchestration/observability.	Creates a short text column. It provides a governed place for trace key to link dataverse updates to future orchestration/observability.	Wrong length or schema names can create hard-to-change metadata.
49	GlobalChoice("aso_communicationstate", "Communication State",	Defines the Communication State column (aso_communicationstate).	Sets IsGlobal = true and references the global choice logical name. In this script the	Creates a choice column that reuses an existing global choice. It provides a	The global choice logical name must already exist and match exactly.

	"aso_communicationstate", "Uses global Communication State choice."),		field purpose is: Reusable communication governance state for the person/contact.	governed place for reusable communication governance state for the person/contact.	
50	GlobalChoice("aso_lifecyclecommunications tage", "Lifecycle Communication Stage", "aso_lifecyclecommunicationstage", "Uses global Lifecycle Communication Stage choice."),	Defines the Lifecycle Communication Stage column (aso_lifecyclecommunicationstage).	Sets IsGlobal = true and references the global choice logical name. In this script the field purpose is: Reusable lifecycle stage for contact communication.	Creates a choice column that reuses an existing global choice. It provides a governed place for reusable lifecycle stage for contact communication.	The global choice logical name must already exist and match exactly.
51	GlobalChoice("aso_journeyparticipationstatus", "Journey Participation Status", "aso_journeyparticipationstatus", "Uses global Journey Participation Status choice."),	Defines the Journey Participation Status column (aso_journeyparticipationstatus).	Sets IsGlobal = true and references the global choice logical name. In this script the field purpose is: Reusable Customer Insights journey participation status.	Creates a choice column that reuses an existing global choice. It provides a governed place for reusable customer insights journey participation status.	The global choice logical name must already exist and match exactly.
53	Text("aso_customerinsightsjourneyid", "Customer Insights Journey ID", 100, "Latest journey reference."),	Defines the Customer Insights Journey ID column (aso_customerinsightsjourneyid).	Uses StringAttributeMetadata with a maximum length. This is suitable for IDs, names, labels, and references. In this script the field purpose is: Latest related Customer Insights journey identifier.	Creates a short text column. It provides a governed place for latest related customer insights journey identifier.	Wrong length or schema names can create hard-to-change metadata.
54	Text("aso_customerinsightsjourneyname", "Customer Insights Journey Name", 200, "Latest journey name."),	Defines the Customer Insights Journey Name column (aso_customerinsightsjourneyname).	Uses StringAttributeMetadata with a maximum length. This is suitable for IDs, names, labels, and references. In this script the field purpose is: Latest related Customer Insights journey name.	Creates a short text column. It provides a governed place for latest related customer insights journey name.	Wrong length or schema names can create hard-to-change metadata.
55	DateTimeField("aso_customerinsightslastentryon", "Customer Insights Last Entry On", "Last entry."),	Defines the Customer Insights Last Entry On column (aso_customerinsightslastentryon).	Uses DateTimeAttributeMetadata with DateTimeFormat.DateAndTime and DateTimeBehavior.UserLocal. In this script the field purpose is: Timestamp for last journey entry.	Creates a date/time column. It provides a governed place for timestamp for last journey entry.	Wrong datetime behavior can confuse users in different time zones.
56	DateTimeField("aso_customerinsightslastinteractionon", "Customer Insights Last Interaction On", "Last interaction."),	Defines the Customer Insights Last Interaction On column (aso_customerinsightslastinteractionon).	Uses DateTimeAttributeMetadata with DateTimeFormat.DateAndTime and DateTimeBehavior.UserLocal. In this script the field purpose is: Timestamp for latest Customer Insights interaction.	Creates a date/time column. It provides a governed place for timestamp for latest customer insights interaction.	Wrong datetime behavior can confuse users in different time zones.
58	LocalChoice("aso_customerinsightslastinteractiontype", "Customer Insights Last Interaction Type",	Defines the Customer Insights Last Interaction Type column (aso_customerinsightslastinteractiontype).	Builds a non-global OptionSetMetadata and attaches local OptionMetadata values to the column. In this script the field purpose is: Type of the latest Customer Insights interaction.	Creates a table-specific choice column. It provides a governed place for type of the latest customer insights interaction.	Do not use local choices for values that must be reused across many tables.
59	new[] { "EmailSent", "Open", "Click", "FormSubmit", "Reply", "Unsubscribe", "Bounce", "CustomAction" },	Defines the option values for the preceding local choice column.	The string array is passed into LocalChoice, which converts each value into OptionMetadata.	These labels become the selectable choices in Dataverse.	Changing option text later may affect integrations, filters, and documentation.
60	"Customer Insights last interaction type."),	Provides the description for the Customer Insights Last Interaction Type choice.	This closes the LocalChoice call after the value array.	The description helps makers understand the field purpose in Power Apps.	Descriptions should remain business-readable and not contradict the data model.
62	GlobalChoice("aso_emailconsentstatus", "Email Consent Status", "aso_emailconsentstatus", "Uses global Email Consent Status choice."),	Defines the Email Consent Status column (aso_emailconsentstatus).	Sets IsGlobal = true and references the global choice logical name. In this script the field purpose is: Reusable consent status for communication eligibility.	Creates a choice column that reuses an existing global choice. It provides a governed place for reusable consent status for communication eligibility.	The global choice logical name must already exist and match exactly.
63	Text("aso_complianceprofilename", "Compliance Profile Name", 200, "Profile used."),	Defines the Compliance Profile Name column (aso_complianceprofilename).	Uses StringAttributeMetadata with a maximum length. This is suitable for IDs, names, labels, and references. In this script the field purpose is: Compliance profile used for Customer Insights communication governance.	Creates a short text column. It provides a governed place for compliance profile used for customer insights communication governance.	Wrong length or schema names can create hard-to-change metadata.
64	Memo("aso_communicationholdreason", "Communication Hold Reason", "Suppression / hold reason.")	Defines the Communication Hold Reason column (aso_communicationholdreason).	Uses MemoAttributeMetadata with MaxLength = 4000 so longer summaries and rationale can be stored. In this script the field purpose is: Reason communication is blocked, suppressed, or held.	Creates a multiline text column. It provides a governed place for reason communication is blocked, suppressed, or held.	Do not use short text for AI summaries or hold reasons because content may be truncated.
65	};	Opens or closes a C# block or object initializer.	Braces and terminators define scope and statement boundaries in C#.	They keep methods, loops, conditionals, and metadata objects syntactically valid.	Missing or extra braces/semicolons cause compile errors or wrong structure.
67	foreach (var field in fields)	Starts a loop over each planned column.	The foreach loop processes every AttributeMetadata object in the fields list.	It avoids repeated manual create logic for each column.	If the fields list is wrong, the loop will faithfully create the wrong metadata.
68	{	Opens or closes a C# block or object initializer.	Braces and terminators define scope and statement boundaries in C#.	They keep methods, loops, conditionals, and metadata objects syntactically valid.	Missing or extra braces/semicolons cause compile errors or wrong structure.
69	var logicalName = field.SchemaName!.ToLowerInvariant();	Gets the schema name of the current field and converts it to lowercase.	Dataverse logical names are lowercase; ToLowerInvariant helps compare planned fields with existing fields.	This supports duplicate detection and reliable skip behavior.	Removing lowercase normalization can cause inconsistent comparisons.
71	if (existingAttributes.Contains(logicalName))	Checks whether the current column already exists.	The HashSet lookup uses case-insensitive comparison of existing attribute logical names.	This makes the script safe to rerun after a partial or previous execution.	If this is missing, duplicate-column errors are likely.
72	{	Opens or closes a C# block or object initializer.	Braces and terminators define scope and statement boundaries in C#.	They keep methods, loops, conditionals, and metadata objects syntactically valid.	Missing or extra braces/semicolons cause compile errors or wrong structure.
73	Console.ForegroundColor = ConsoleColor.Yellow;	Changes terminal output color to yellow for skipped items.	Yellow is used when a column already exists and is intentionally skipped.	This shows the script is idempotent and safe to rerun.	A skip is not usually an error; it means the field already exists.

74	Console.WriteLine(\$"SKIP existing: {logicalName}");	Prints that the current column was skipped because it already exists.	The message includes the logical name for traceability.	Operators can distinguish safe skip behavior from failure.	Skipped fields should still be validated if data types are uncertain.
75	Console.ResetColor();	Resets terminal text color back to normal.	Console.ResetColor clears the previous ForegroundColor setting.	It keeps terminal output readable after colored status lines.	If missing, later output may stay red/yellow/green.
76	continue;	Moves to the next field in the loop.	continue skips the create request for an existing column.	This prevents the script from trying to recreate an existing column.	Without it, the script would continue into the create block even after detecting duplicates.
77	}	Opens or closes a C# block or object initializer.	Braces and terminators define scope and statement boundaries in C#.	They keep methods, loops, conditionals, and metadata objects syntactically valid.	Missing or extra braces/semicolons cause compile errors or wrong structure.
79	try	Begins a protected block for creating a column.	C# try/catch captures exceptions from Dataverse SDK requests.	One failed field can be reported clearly without hiding the cause.	Do not suppress errors silently; the catch block must print them.
80	{	Opens or closes a C# block or object initializer.	Braces and terminators define scope and statement boundaries in C#.	They keep methods, loops, conditionals, and metadata objects syntactically valid.	Missing or extra braces/semicolons cause compile errors or wrong structure.
81	Console.WriteLine(\$"Creating: {logicalName} ...");	Prints which field is about to be created.	The log includes the logical name of the current field.	It provides an execution trace for customer IT and troubleshooting.	If the script stops, this helps identify the last attempted field.
83	var request = new CreateAttributeRequest	Creates the Dataverse request object for creating a column.	CreateAttributeRequest is the SDK metadata operation for adding an attribute to an existing table.	This is the core create-column mechanism.	Using the wrong request type will not create table columns.
84	{	Opens or closes a C# block or object initializer.	Braces and terminators define scope and statement boundaries in C#.	They keep methods, loops, conditionals, and metadata objects syntactically valid.	Missing or extra braces/semicolons cause compile errors or wrong structure.
85	EntityName = EntityLogicalName,	Sets the table that receives the new column.	EntityName receives account or contact from the constant.	It ensures the current attribute is created on the intended Dataverse table.	Wrong EntityName creates fields on the wrong table or fails.
86	Attribute = field,	Provides the current column metadata to the create request.	The Attribute property receives a StringAttributeMetadata, MemoAttributeMetadata, PicklistAttributeMetadata, or DateTimeAttributeMetadata object.	This tells Dataverse exactly what type of column to create.	If field metadata is malformed, the create request fails.
87	SolutionUniqueName = SolutionUniqueName	Associates the created column with ASO.Core.	The request property instructs Dataverse to add the metadata component to the specified solution.	This is essential for ALM, export, and customer handover.	If omitted or wrong, fields may not appear in the expected solution.
88	};	Opens or closes a C# block or object initializer.	Braces and terminators define scope and statement boundaries in C#.	They keep methods, loops, conditionals, and metadata objects syntactically valid.	Missing or extra braces/semicolons cause compile errors or wrong structure.
90	service.Execute(request);	Sends the create-column request to Dataverse.	ServiceClient.Execute calls the Dataverse organization service with the SDK request.	This is the line that actually creates the missing column.	Requires sufficient privileges and valid metadata definitions.
92	Console.ForegroundColor = ConsoleColor.Green;	Changes terminal output color to green for successful actions.	Green is used after successful creation or publishing.	This gives an easy visual confirmation during execution.	Green output should still be validated in Power Apps.
93	Console.WriteLine(\$"CREATED: {logicalName}");	Prints a success message for the created column.	The message includes the logical name returned from the current field metadata.	It creates a readable deployment log.	Still validate in Power Apps because terminal success is not a substitute for solution review.
94	Console.ResetColor();	Resets terminal text color back to normal.	Console.ResetColor clears the previous ForegroundColor setting.	It keeps terminal output readable after colored status lines.	If missing, later output may stay red/yellow/green.
95	}	Opens or closes a C# block or object initializer.	Braces and terminators define scope and statement boundaries in C#.	They keep methods, loops, conditionals, and metadata objects syntactically valid.	Missing or extra braces/semicolons cause compile errors or wrong structure.
96	catch (Exception ex)	Starts error handling for a failed column creation.	Any exception from Dataverse or the SDK is captured in ex.	The script can show the exact failing field and error message.	Do not ignore caught errors; fix and rerun.
97	{	Opens or closes a C# block or object initializer.	Braces and terminators define scope and statement boundaries in C#.	They keep methods, loops, conditionals, and metadata objects syntactically valid.	Missing or extra braces/semicolons cause compile errors or wrong structure.
98	Console.ForegroundColor = ConsoleColor.Red;	Changes terminal output color to red for errors.	Console.ForegroundColor controls text color in the terminal.	Operators can quickly identify failures.	Always reset color afterward to avoid confusing output.
99	Console.WriteLine(\$"FAILED: {logicalName}");	Prints which field failed.	The field logical name is included for diagnosis.	This helps IT identify the exact metadata issue.	Common causes include wrong global choice name, permissions, or duplicate field with different type.
100	Console.WriteLine(ex.Message);	Prints the technical error message.	ex.Message contains the SDK or Dataverse exception description.	This is needed for troubleshooting.	Do not remove it; otherwise failures become hard to diagnose.
101	Console.ResetColor();	Resets terminal text color back to normal.	Console.ResetColor clears the previous ForegroundColor setting.	It keeps terminal output readable after colored status lines.	If missing, later output may stay red/yellow/green.
102	}	Opens or closes a C# block or object initializer.	Braces and terminators define scope and statement boundaries in C#.	They keep methods, loops, conditionals, and metadata objects syntactically valid.	Missing or extra braces/semicolons cause compile errors or wrong structure.
103	}	Opens or closes a C# block or object initializer.	Braces and terminators define scope and statement boundaries in C#.	They keep methods, loops, conditionals, and metadata objects syntactically valid.	Missing or extra braces/semicolons cause compile errors or wrong structure.
105	Console.WriteLine();	Writes a blank line to the terminal.	This is a formatting line only.	It makes logs easier to read.	No business or Dataverse effect.
106	Console.WriteLine("Publishing Account table customizations...");	Prints that Account customizations are being published.	The script is about to execute PublishXmlRequest for the target table.	Publishing makes metadata changes visible and usable in the maker/runtime experience.	If publishing fails, fields may exist but not be fully available.
108	service.Execute(new PublishXmlRequest	Starts the targeted publish request.	PublishXmlRequest publishes selected Dataverse components rather than all customizations.	It activates the table metadata changes after creation.	Publishing all customizations is broader; targeted publish is cleaner for this script.
109	{	Opens or closes a C# block or object initializer.	Braces and terminators define scope and statement boundaries in C#.	They keep methods, loops, conditionals, and metadata objects syntactically valid.	Missing or extra braces/semicolons cause compile errors or wrong structure.
110	ParameterXml = "<importexportxml><entities><entity>account</entity></entities></importexportxml>"	Specifies that only the account table should be published.	The XML tells Dataverse which entity metadata to publish.	It avoids unnecessary publishing of unrelated components.	Wrong XML can publish the wrong table or fail the request.
111	});	Defines part of the C# script structure.	This line contributes to the .NET console application that creates Dataverse metadata.	It supports repeatable and auditable table extension work.	Changing it without understanding the SDK behavior can cause build or metadata errors.
113	Console.ForegroundColor =	Changes terminal output color to green for	Green is used after successful creation or	This gives an easy visual confirmation	Green output should still be validated in

		successful actions.	publishing.	during execution.	Power Apps.
114	Console.WriteLine("Done. Validate in ASO.Core → Tables → Account → Columns.");	Prints the final validation instruction.	The script ends by reminding the operator to check Power Apps.	Human validation remains mandatory after metadata deployment.	Never skip the validation step before backup or next implementation step.
115	Console.ResetColor();	Resets terminal text color back to normal.	Console.ResetColor clears the previous ForegroundColor setting.	It keeps terminal output readable after colored status lines.	If missing, later output may stay red/yellow/green.
117	static HashSet<string> GetExistingAttributeLogicalNames(ServiceClient service, string entityLogicalName)	Defines a helper function to retrieve existing column logical names.	Returns a HashSet<string> for fast, case-insensitive duplicate checks.	This enables safe reruns and partial recovery.	If this helper is wrong, duplicate detection fails.
118	{	Opens or closes a C# block or object initializer.	Braces and terminators define scope and statement boundaries in C#.	They keep methods, loops, conditionals, and metadata objects syntactically valid.	Missing or extra braces/semicolons cause compile errors or wrong structure.
119	var response = (RetrieveEntityResponse)service.Execute(new RetrieveEntityRequest	Creates or executes a request to retrieve table metadata.	RetrieveEntityRequest asks Dataverse for metadata about a table.	The script needs current attributes before creating new ones.	Incorrect logical names or missing privileges can cause this request to fail.
120	{	Opens or closes a C# block or object initializer.	Braces and terminators define scope and statement boundaries in C#.	They keep methods, loops, conditionals, and metadata objects syntactically valid.	Missing or extra braces/semicolons cause compile errors or wrong structure.
121	LogicalName = entityLogicalName,	Tells the metadata retrieval request which table to inspect.	The helper receives account/contact and assigns it to the request LogicalName.	It retrieves attributes for the correct table.	Wrong table logical name gives wrong duplicate-detection results.
122	EntityFilters = EntityFilters.Attributes,	Requests only attribute/column metadata.	EntityFilters.Attributes limits the response to column definitions.	The script only needs the list of existing columns.	Using the wrong filter may return insufficient metadata.
123	RetrieveAsIfPublished = true	Reads metadata as if unpublished changes are already visible.	This includes latest customizations even before full publication.	It improves duplicate detection during iterative development.	Without it, recent unpublished fields might be missed.
124	});	Defines part of the C# script structure.	This line contributes to the .NET console application that creates Dataverse metadata.	It supports repeatable and auditable table extension work.	Changing it without understanding the SDK behavior can cause build or metadata errors.
126	return response.EntityMetadata.Attributes	Starts returning the existing attribute collection.	The function chains LINQ operations over EntityMetadata.Attributes.	It transforms Dataverse metadata into a usable set of logical names.	If EntityMetadata is null, retrieval failed earlier.
127	.Where(a => ! string.IsNullOrEmpty(a.LogicalName))	Filters out attributes without usable logical names.	LINQ Where removes null, empty, or whitespace logical names.	It avoids invalid values in the duplicate-detection set.	Do not compare null logical names.
128	.Select(a => a.LogicalName!)	Selects only the logical name from each attribute.	The null-forgiving operator tells C# the value is expected not to be null after filtering.	The script only needs names for comparison.	Do not use display names for metadata comparisons.
129	.ToHashSet(StringComparer.OrdinalgnoreCase);	Creates a case-insensitive set of existing logical names.	HashSet gives fast Contains checks and ignores case differences.	It supports safe and efficient skip logic.	Case-sensitive checks could miss existing fields.
130	}	Opens or closes a C# block or object initializer.	Braces and terminators define scope and statement boundaries in C#.	They keep methods, loops, conditionals, and metadata objects syntactically valid.	Missing or extra braces/semicolons cause compile errors or wrong structure.
132	static Label Label(string value) => new(value, LanguageCode);	Defines a helper to create Dataverse labels.	It returns a Label object with the configured LanguageCode.	All display names and descriptions need Dataverse Label objects.	Raw strings are not accepted for DisplayName and Description metadata.
134	static AttributeRequiredLevelManagedProperty Optional()	Defines a helper for optional fields.	It returns AttributeRequiredLevel.None wrapped in a managed property object.	The ASO extension fields are optional at schema level for MVP safety.	Making fields required can break record creation or existing processes.
135	{	Opens or closes a C# block or object initializer.	Braces and terminators define scope and statement boundaries in C#.	They keep methods, loops, conditionals, and metadata objects syntactically valid.	Missing or extra braces/semicolons cause compile errors or wrong structure.
136	return new AttributeRequiredLevelManagedProperty(AttributeRequiredLevel.None);	Marks a column as not required.	Dataverse uses AttributeRequiredLevelManagedProperty to define requirement behavior.	MVP fields should not block users or integrations when values are not yet available.	Do not make AI/SAP/CI fields required unless business process is mature.
137	}	Opens or closes a C# block or object initializer.	Braces and terminators define scope and statement boundaries in C#.	They keep methods, loops, conditionals, and metadata objects syntactically valid.	Missing or extra braces/semicolons cause compile errors or wrong structure.
139	static StringAttributeMetadata Text(string schemaName, string displayName, int maxLength, string description)	Defines the helper that creates text-column metadata.	The helper returns a StringAttributeMetadata object for Dataverse short text fields.	It avoids repeating the same metadata boilerplate for every text field.	MaxLength must be appropriate for the expected value.
140	{	Opens or closes a C# block or object initializer.	Braces and terminators define scope and statement boundaries in C#.	They keep methods, loops, conditionals, and metadata objects syntactically valid.	Missing or extra braces/semicolons cause compile errors or wrong structure.
141	return new StringAttributeMetadata	Returns a new short text metadata object.	This object describes a Dataverse text column to the SDK.	The create request uses this object to create a text field.	If properties are missing, the request may fail or create incomplete metadata.
142	{	Opens or closes a C# block or object initializer.	Braces and terminators define scope and statement boundaries in C#.	They keep methods, loops, conditionals, and metadata objects syntactically valid.	Missing or extra braces/semicolons cause compile errors or wrong structure.
143	SchemaName = schemaName,	Sets the schema name of the column.	Dataverse uses schema/logical names as permanent technical identifiers.	Correct names are essential for ALM, integrations, Power Automate, and forms.	Schema names are difficult or impossible to rename later.
144	DisplayName = Label(displayName),	Sets the human-readable column name.	The helper converts the string into a Dataverse Label object.	This is what makers and users see in the UI.	Display names can be changed later, but schema names should remain stable.
145	Description = Label(description),	Sets the field description.	Descriptions are stored as Dataverse labels and help makers understand the field purpose.	Good descriptions reduce confusion for customer IT and future maintainers.	Vague descriptions reduce handover quality.
146	RequiredLevel = Optional(),	Marks the field as optional.	Calls the Optional helper to return an AttributeRequiredLevelManagedProperty.	Optional fields do not block existing sales/customer records.	Required fields can break existing forms, imports, and integrations.
147	MaxLength = maxLength	Sets the maximum length for short text fields.	The value comes from the helper argument, such as 100 or 200.	It controls how much text Dataverse stores.	Too small a value truncates data; too large can be unnecessary.
148	};	Opens or closes a C# block or object initializer.	Braces and terminators define scope and statement boundaries in C#.	They keep methods, loops, conditionals, and metadata objects syntactically valid.	Missing or extra braces/semicolons cause compile errors or wrong structure.
149	}	Opens or closes a C# block or object initializer.	Braces and terminators define scope and statement boundaries in C#.	They keep methods, loops, conditionals, and metadata objects syntactically valid.	Missing or extra braces/semicolons cause compile errors or wrong structure.
151	static MemoAttributeMetadata Memo(string schemaName, string displayName, string description)	Defines the helper that creates multiline text-column metadata.	The helper returns MemoAttributeMetadata with a 4000 character maximum.	It standardizes long text fields used for AI summaries and hold reasons.	Memo fields are heavier than text fields; use only when needed.

152	{	Opens or closes a C# block or object initializer.	Braces and terminators define scope and statement boundaries in C#.	They keep methods, loops, conditionals, and metadata objects syntactically valid.	Missing or extra braces/semicolons cause compile errors or wrong structure.
153	return new MemoAttributeMetadata	Returns a new multiline text metadata object.	This object describes a Dataverse memo column.	The create request uses this object to create a long text field.	Large text fields should be used intentionally.
154	{	Opens or closes a C# block or object initializer.	Braces and terminators define scope and statement boundaries in C#.	They keep methods, loops, conditionals, and metadata objects syntactically valid.	Missing or extra braces/semicolons cause compile errors or wrong structure.
155	SchemaName = schemaName,	Sets the schema name of the column.	Dataverse uses schema/logical names as permanent technical identifiers.	Correct names are essential for ALM, integrations, Power Automate, and forms.	Schema names are difficult or impossible to rename later.
156	DisplayName = Label(displayName),	Sets the human-readable column name.	The helper converts the string into a Dataverse Label object.	This is what makers and users see in the UI.	Display names can be changed later, but schema names should remain stable.
157	Description = Label(description),	Sets the field description.	Descriptions are stored as Dataverse labels and help makers understand the field purpose.	Good descriptions reduce confusion for customer IT and future maintainers.	Vague descriptions reduce handover quality.
158	RequiredLevel = Optional(),	Marks the field as optional.	Calls the Optional helper to return an AttributeRequiredLevelManagedProperty.	Optional fields do not block existing sales/customer records.	Required fields can break existing forms, imports, and integrations.
159	MaxLength = 4000	Sets multiline text capacity to 4000 characters.	MemoAttributeMetadata supports larger text values than StringAttributeMetadata.	AI summaries and hold reasons may need paragraphs of text.	Very large text fields should not be overused in views.
160	};	Opens or closes a C# block or object initializer.	Braces and terminators define scope and statement boundaries in C#.	They keep methods, loops, conditionals, and metadata objects syntactically valid.	Missing or extra braces/semicolons cause compile errors or wrong structure.
161	}	Opens or closes a C# block or object initializer.	Braces and terminators define scope and statement boundaries in C#.	They keep methods, loops, conditionals, and metadata objects syntactically valid.	Missing or extra braces/semicolons cause compile errors or wrong structure.
163	static DateTimeAttributeMetadata DateTimeField(string schemaName, string displayName, string description)	Defines the helper that creates date/time-column metadata.	The helper returns DateTimeAttributeMetadata with user-local behavior.	It standardizes timestamps across ASO fields.	Check timezone behavior for integration scenarios.
164	{	Opens or closes a C# block or object initializer.	Braces and terminators define scope and statement boundaries in C#.	They keep methods, loops, conditionals, and metadata objects syntactically valid.	Missing or extra braces/semicolons cause compile errors or wrong structure.
165	return new DateTimeAttributeMetadata	Returns a new date/time metadata object.	This object describes a Dataverse date and time column.	It supports storing Customer Insights and AI run timestamps.	Wrong behavior setting can affect time display.
166	{	Opens or closes a C# block or object initializer.	Braces and terminators define scope and statement boundaries in C#.	They keep methods, loops, conditionals, and metadata objects syntactically valid.	Missing or extra braces/semicolons cause compile errors or wrong structure.
167	SchemaName = schemaName,	Sets the schema name of the column.	Dataverse uses schema/logical names as permanent technical identifiers.	Correct names are essential for ALM, integrations, Power Automate, and forms.	Schema names are difficult or impossible to rename later.
168	DisplayName = Label(displayName),	Sets the human-readable column name.	The helper converts the string into a Dataverse Label object.	This is what makers and users see in the UI.	Display names can be changed later, but schema names should remain stable.
169	Description = Label(description),	Sets the field description.	Descriptions are stored as Dataverse labels and help makers understand the field purpose.	Good descriptions reduce confusion for customer IT and future maintainers.	Vague descriptions reduce handover quality.
170	RequiredLevel = Optional(),	Marks the field as optional.	Calls the Optional helper to return an AttributeRequiredLevelManagedProperty.	Optional fields do not block existing sales/customer records.	Required fields can break existing forms, imports, and integrations.
171	Format = DateTimeFormat.DateAndTime,	Configures the field to store date and time, not date-only.	DateTimeFormat.DateAndTime includes both date and clock time.	Run timestamps and interaction timestamps require time precision.	Date-only would lose important event timing.
172	DateTimeBehavior = DateTimeBehavior.UserLocal	Displays the time according to the user local timezone behavior.	UserLocal is common for user-facing activity and interaction timestamps.	Business users see times in their expected local context.	Integration designs should confirm whether UserLocal or TimeZoneIndependent is better.
173	};	Opens or closes a C# block or object initializer.	Braces and terminators define scope and statement boundaries in C#.	They keep methods, loops, conditionals, and metadata objects syntactically valid.	Missing or extra braces/semicolons cause compile errors or wrong structure.
174	}	Opens or closes a C# block or object initializer.	Braces and terminators define scope and statement boundaries in C#.	They keep methods, loops, conditionals, and metadata objects syntactically valid.	Missing or extra braces/semicolons cause compile errors or wrong structure.
176	static PicklistAttributeMetadata LocalChoice(string schemaName, string displayName, string[] values, string description)	Defines the helper for local choice columns.	It creates a PicklistAttributeMetadata object with an embedded non-global OptionSetMetadata.	It keeps table-specific values local when they are not reusable global choices.	Local choices are harder to reuse across tables.
177	{	Opens or closes a C# block or object initializer.	Braces and terminators define scope and statement boundaries in C#.	They keep methods, loops, conditionals, and metadata objects syntactically valid.	Missing or extra braces/semicolons cause compile errors or wrong structure.
178	var optionSet = new OptionSetMetadata	Creates a new option set object for a local choice.	This object stores the choice metadata before it is attached to the picklist field.	Local choices need an embedded option set definition.	This is different from referencing a global choice.
179	{	Opens or closes a C# block or object initializer.	Braces and terminators define scope and statement boundaries in C#.	They keep methods, loops, conditionals, and metadata objects syntactically valid.	Missing or extra braces/semicolons cause compile errors or wrong structure.
180	IsGlobal = false,	Marks the option set as local to the column.	The choices are stored with the field, not as a reusable global component.	It is appropriate for values specific to one column.	Do not use local choices when many tables need the same values.
181	OptionSetType = OptionSetType.Picklist	Defines the choice type as a single-select picklist.	Picklist means one value can be selected for the column.	The ASO status and interaction fields expect one current value.	Use multi-select only when the business model requires multiple values.
182	};	Opens or closes a C# block or object initializer.	Braces and terminators define scope and statement boundaries in C#.	They keep methods, loops, conditionals, and metadata objects syntactically valid.	Missing or extra braces/semicolons cause compile errors or wrong structure.
184	foreach (var value in values)	Loops over every local choice label.	Each string from the array is converted into an OptionMetadata object.	It avoids writing repeated option-add code manually.	The array values become metadata labels; review spelling before running.
185	{	Opens or closes a C# block or object initializer.	Braces and terminators define scope and statement boundaries in C#.	They keep methods, loops, conditionals, and metadata objects syntactically valid.	Missing or extra braces/semicolons cause compile errors or wrong structure.
186	optionSet.Options.Add(new OptionMetadata(Label(value), null));	Adds one option value to the local choice.	OptionMetadata contains the label and lets Dataverse assign the numeric value automatically.	It creates the actual selectable values.	Do not manually guess option values unless you have an ALM reason.
187	}	Opens or closes a C# block or object initializer.	Braces and terminators define scope and statement boundaries in C#.	They keep methods, loops, conditionals, and metadata objects syntactically valid.	Missing or extra braces/semicolons cause compile errors or wrong structure.
189	return new PicklistAttributeMetadata	Returns a new choice metadata object.	The object describes either a local choice or	It enables governed option values rather	Choice configuration must match the data

			global-choice-backed column.	than free text.	model.
190	{	Opens or closes a C# block or object initializer.	Braces and terminators define scope and statement boundaries in C#.	They keep methods, loops, conditionals, and metadata objects syntactically valid.	Missing or extra braces/semicolons cause compile errors or wrong structure.
191	SchemaName = schemaName,	Sets the schema name of the column.	Dataverse uses schema/logical names as permanent technical identifiers.	Correct names are essential for ALM, integrations, Power Automate, and forms.	Schema names are difficult or impossible to rename later.
192	DisplayName = Label(displayName),	Sets the human-readable column name.	The helper converts the string into a Dataverse Label object.	This is what makers and users see in the UI.	Display names can be changed later, but schema names should remain stable.
193	Description = Label(description),	Sets the field description.	Descriptions are stored as Dataverse labels and help makers understand the field purpose.	Good descriptions reduce confusion for customer IT and future maintainers.	Vague descriptions reduce handover quality.
194	RequiredLevel = Optional(),	Marks the field as optional.	Calls the Optional helper to return an AttributeRequiredLevelManagedProperty.	Optional fields do not block existing sales/customer records.	Required fields can break existing forms, imports, and integrations.
195	OptionSet = optionSet	Attaches the local option set to the choice column.	The PicklistAttributeMetadata receives the option set created earlier.	Without this, the choice field would have no values.	Missing option sets cause invalid or empty choice definitions.
196	};	Opens or closes a C# block or object initializer.	Braces and terminators define scope and statement boundaries in C#.	They keep methods, loops, conditionals, and metadata objects syntactically valid.	Missing or extra braces/semicolons cause compile errors or wrong structure.
197	}	Opens or closes a C# block or object initializer.	Braces and terminators define scope and statement boundaries in C#.	They keep methods, loops, conditionals, and metadata objects syntactically valid.	Missing or extra braces/semicolons cause compile errors or wrong structure.
199	static PicklistAttributeMetadata GlobalChoice(string schemaName, string displayName, string globalChoiceName, string description)	Defines the helper for global-choice-backed columns.	It creates PicklistAttributeMetadata that references an existing global option set by name.	It enforces consistent taxonomy across Account, Contact, Lead, and Opportunity.	The global choice must exist before running the script.
200	{	Opens or closes a C# block or object initializer.	Braces and terminators define scope and statement boundaries in C#.	They keep methods, loops, conditionals, and metadata objects syntactically valid.	Missing or extra braces/semicolons cause compile errors or wrong structure.
201	return new PicklistAttributeMetadata	Returns a new choice metadata object.	The object describes either a local choice or global-choice-backed column.	It enables governed option values rather than free text.	Choice configuration must match the data model.
202	{	Opens or closes a C# block or object initializer.	Braces and terminators define scope and statement boundaries in C#.	They keep methods, loops, conditionals, and metadata objects syntactically valid.	Missing or extra braces/semicolons cause compile errors or wrong structure.
203	SchemaName = schemaName,	Sets the schema name of the column.	Dataverse uses schema/logical names as permanent technical identifiers.	Correct names are essential for ALM, integrations, Power Automate, and forms.	Schema names are difficult or impossible to rename later.
204	DisplayName = Label(displayName),	Sets the human-readable column name.	The helper converts the string into a Dataverse Label object.	This is what makers and users see in the UI.	Display names can be changed later, but schema names should remain stable.
205	Description = Label(description),	Sets the field description.	Descriptions are stored as Dataverse labels and help makers understand the field purpose.	Good descriptions reduce confusion for customer IT and future maintainers.	Vague descriptions reduce handover quality.
206	RequiredLevel = Optional(),	Marks the field as optional.	Calls the Optional helper to return an AttributeRequiredLevelManagedProperty.	Optional fields do not block existing sales/customer records.	Required fields can break existing forms, imports, and integrations.
207	OptionSet = new OptionSetMetadata	Defines part of the C# script structure.	This line contributes to the .NET console application that creates Dataverse metadata.	It supports repeatable and auditable table extension work.	Changing it without understanding the SDK behavior can cause build or metadata errors.
208	{	Opens or closes a C# block or object initializer.	Braces and terminators define scope and statement boundaries in C#.	They keep methods, loops, conditionals, and metadata objects syntactically valid.	Missing or extra braces/semicolons cause compile errors or wrong structure.
209	IsGlobal = true,	Marks the option set reference as global.	The field points to an existing reusable global choice component.	This keeps communication and consent states consistent across tables.	The referenced global choice must already exist.
210	Name = globalChoiceName,	Sets the logical name of the existing global choice to reuse.	Dataverse resolves this name to a global option set component.	This ties the column to the governed shared taxonomy.	A typo here causes a global choice not found error.
211	OptionSetType = OptionSetType.Picklist	Defines the choice type as a single-select picklist.	Picklist means one value can be selected for the column.	The ASO status and interaction fields expect one current value.	Use multi-select only when the business model requires multiple values.
212	}	Opens or closes a C# block or object initializer.	Braces and terminators define scope and statement boundaries in C#.	They keep methods, loops, conditionals, and metadata objects syntactically valid.	Missing or extra braces/semicolons cause compile errors or wrong structure.
213	};	Opens or closes a C# block or object initializer.	Braces and terminators define scope and statement boundaries in C#.	They keep methods, loops, conditionals, and metadata objects syntactically valid.	Missing or extra braces/semicolons cause compile errors or wrong structure.
214	}	Opens or closes a C# block or object initializer.	Braces and terminators define scope and statement boundaries in C#.	They keep methods, loops, conditionals, and metadata objects syntactically valid.	Missing or extra braces/semicolons cause compile errors or wrong structure.

Contact script - line-by-line explanation

Line	Code	Explanation in simple language	Technical explanation	Why it matters	Common mistake / warning
1	using Microsoft.Crm.Sdk.Messages;	Imports Microsoft.Crm.Sdk.Messages so the script can use Dataverse and C# SDK classes.	C# namespaces provide the types used later, such as ServiceClient, CreateAttributeRequest, RetrieveEntityRequest, and metadata classes.	Without the correct imports, the compiler cannot resolve SDK types.	Removing required using statements causes compile errors such as type or namespace not found.
2	using Microsoft.PowerPlatform.Dataverse.Client;	Imports Microsoft.PowerPlatform.Dataverse.Client so the script can use Dataverse and C# SDK classes.	C# namespaces provide the types used later, such as ServiceClient, CreateAttributeRequest, RetrieveEntityRequest, and metadata classes.	Without the correct imports, the compiler cannot resolve SDK types.	Removing required using statements causes compile errors such as type or namespace not found.
3	using Microsoft.Xrm.Sdk;	Imports Microsoft.Xrm.Sdk so the script can	C# namespaces provide the types used	Without the correct imports, the compiler	Removing required using statements causes

		use Dataverse and C# SDK classes.	later, such as ServiceClient, CreateAttributeRequest, RetrieveEntityRequest, and metadata classes.	cannot resolve SDK types.	compile errors such as type or namespace not found.
4	using Microsoft.Xrm.Sdk.Messages;	Imports Microsoft.Xrm.Sdk.Messages so the script can use Dataverse and C# SDK classes.	C# namespaces provide the types used later, such as ServiceClient, CreateAttributeRequest, RetrieveEntityRequest, and metadata classes.	Without the correct imports, the compiler cannot resolve SDK types.	Removing required using statements causes compile errors such as type or namespace not found.
5	using Microsoft.Xrm.Sdk.Metadata;	Imports Microsoft.Xrm.Sdk.Metadata so the script can use Dataverse and C# SDK classes.	C# namespaces provide the types used later, such as ServiceClient, CreateAttributeRequest, RetrieveEntityRequest, and metadata classes.	Without the correct imports, the compiler cannot resolve SDK types.	Removing required using statements causes compile errors such as type or namespace not found.
7	const string DataverseUrl = "https://phoenicarix-ci.crm4.dynamics.com";	Stores the target Dataverse environment URL: Phoenicarix-CI.	The ServiceClient connection string later reads this constant to decide which Dataverse organization receives metadata changes.	It prevents the script from accidentally targeting the wrong environment when reviewed before execution.	A wrong URL creates schema in the wrong environment or fails authentication.
8	const string SolutionUniqueName = "ASOCore";	Stores the solution unique name ASOCore.	CreateAttributeRequest and CreateEntityRequest can use SolutionUniqueName to associate created components with a solution layer.	This keeps columns inside ASO.Core instead of unmanaged/default solution context only.	Using the display name instead of unique name, or a typo, can leave components outside the intended solution.
9	const string EntityLogicalName = "contact";	Stores the target Dataverse table logical name: contact.	Dataverse SDK calls use logical names, not localized display names. This tells the script which standard table to extend.	It ensures the script extends Contact, not another table.	Using a display name like Account or Kontakt instead of account/contact causes SDK failures.
10	const int LanguageCode = 1033;	Sets language code 1033 for labels.	1033 is English. Dataverse labels and descriptions are localized; this selects the label language for DisplayName and Description.	It makes generated labels readable and consistent in the metadata.	If multilingual labels are required, additional localized labels must be added later.
12	var connectionString =	Begins building the Dataverse connection string.	The following lines provide OAuth authentication parameters for ServiceClient.	The script must authenticate before it can read or create Dataverse metadata.	Incomplete connection strings lead to connection failures.
13	\$@"AuthType=OAuth;	Uses OAuth authentication.	OAuth is the authentication type used by ServiceClient for interactive Microsoft Identity sign-in.	It allows the maker/admin user to sign in securely from macOS.	Hardcoding passwords is not acceptable; OAuth is safer for this MVP script.
14	Url={DataverseUrl};	Injects the Dataverse URL into the connection string.	The interpolated connection string substitutes DataverseUrl at runtime.	This keeps the environment target centralized in one constant.	Do not replace this with a different environment unless intentionally redeploying.
15	LoginPrompt=Auto;	Allows the login prompt to appear automatically if needed.	ServiceClient can open an interactive browser/device login when no valid token is cached.	This is convenient for local script execution on a Mac.	In CI/CD this should be replaced with non-interactive authentication.
16	ClientId=51f81489-12ee-4a9e-aaae-a2591f45987d;	Provides the public client ID used for interactive Dataverse tooling authentication.	The GUID is a Microsoft tooling client ID commonly used in Dataverse SDK samples for local OAuth login.	It allows ServiceClient to authenticate without creating a custom app registration for the trial MVP.	For production automation, use an approved application registration and secret/certificate strategy.
17	RedirectUri=http://localhost";	Sets localhost as the OAuth redirect URI.	After browser login, the authentication response returns to the local process.	This enables interactive login to complete on the developer machine.	Changing this can break the local login flow.
19	using var service = new ServiceClient(connectionString);	Imports var service = new ServiceClient(connectionString) so the script can use Dataverse and C# SDK classes.	C# namespaces provide the types used later, such as ServiceClient, CreateAttributeRequest, RetrieveEntityRequest, and metadata classes.	Without the correct imports, the compiler cannot resolve SDK types.	Removing required using statements causes compile errors such as type or namespace not found.
21	if (!service.IsReady)	Checks whether the Dataverse connection is ready.	ServiceClient exposes IsReady to confirm authentication and connection initialization succeeded.	It prevents schema changes from running against a failed or partial connection.	Skipping this check can hide authentication errors until later SDK calls fail.
22	{	Opens or closes a C# block or object initializer.	Braces and terminators define scope and statement boundaries in C#.	They keep methods, loops, conditionals, and metadata objects syntactically valid.	Missing or extra braces/semicolons cause compile errors or wrong structure.
23	Console.ForegroundColor = ConsoleColor.Red;	Changes terminal output color to red for errors.	Console.ForegroundColor controls text color in the terminal.	Operators can quickly identify failures.	Always reset color afterward to avoid confusing output.
24	Console.WriteLine("Connection failed:");	Prints a clear connection failure heading.	This message is shown only inside the failed connection branch.	It tells the operator to fix authentication or environment targeting before continuing.	Do not ignore this message; the script stops after it.
25	Console.WriteLine(service.LastError);	Prints the detailed connection error.	ServiceClient.LastError contains the most recent connection or request error message.	It gives actionable troubleshooting information.	The error may contain authentication or permission details useful for IT.
26	Console.ResetColor();	Resets terminal text color back to normal.	Console.ResetColor clears the previous ForegroundColor setting.	It keeps terminal output readable after colored status lines.	If missing, later output may stay red/yellow/green.
27	return;	Stops the script immediately.	return exits the current top-level program flow.	It prevents metadata operations after connection failure.	Removing it may allow the script to continue with an unusable service object.
28	}	Opens or closes a C# block or object initializer.	Braces and terminators define scope and statement boundaries in C#.	They keep methods, loops, conditionals, and metadata objects syntactically valid.	Missing or extra braces/semicolons cause compile errors or wrong structure.
30	Console.WriteLine(\$"Connected to: {DataverseUrl}");	Prints the connected environment URL.	String interpolation injects DataverseUrl into terminal output.	The operator can visually confirm the target environment before changes are made.	If this shows the wrong URL, stop before continuing.
31	Console.WriteLine(\$"Target solution: {SolutionUniqueName}");	Prints the target solution unique name.	This outputs ASOCore from the constant.	It confirms the target solution layer used for the new metadata.	If it is wrong, fields may not appear in ASO.Core.
32	Console.WriteLine(\$"Target table: {EntityLogicalName}");	Prints the target table logical name.	This outputs account or contact from EntityLogicalName.	It confirms whether the script is extending Account or Contact.	Stop if the table is not the intended one.
33	Console.WriteLine();	Writes a blank line to the terminal.	This is a formatting line only.	It makes logs easier to read.	No business or Dataverse effect.

35	<code>var existingAttributes = GetExistingAttributeLogicalNames(service, EntityLogicalName);</code>	Reads the columns that already exist on the target table.	Calls a helper that executes <code>RetrieveEntityRequest</code> and returns current attribute logical names.	This allows the script to skip existing fields safely.	Without this, rerunning the script would likely fail on duplicate schema names.
37	<code>var fields = new List<AttributeMetadata></code>	Starts the list of columns the script intends to create.	<code>AttributeMetadata</code> is the base class for Dataverse column metadata definitions.	This list is the implementation data contract for the table extension.	Every field added here should be reviewed before execution.
38	<code>{</code>	Opens or closes a C# block or object initializer.	Braces and terminators define scope and statement boundaries in C#.	They keep methods, loops, conditionals, and metadata objects syntactically valid.	Missing or extra braces/semicolons cause compile errors or wrong structure.
39	<code>GlobalChoice("aso_communicationstate", "Communication State", "aso_communicationstate", "Uses global Communication State choice.");</code>	Defines the Communication State column (<code>aso_communicationstate</code>).	Sets <code>IsGlobal</code> = true and references the global choice logical name. In this script the field purpose is: Reusable communication governance state for the person/contact.	Creates a choice column that reuses an existing global choice. It provides a governed place for reusable communication governance state for the person/contact.	The global choice logical name must already exist and match exactly.
40	<code>GlobalChoice("aso_lifecyclecommunications tage", "Lifecycle Communication Stage", "aso_lifecyclecommunicationstage", "Uses global Lifecycle Communication Stage choice.");</code>	Defines the Lifecycle Communication Stage column (<code>aso_lifecyclecommunicationstage</code>).	Sets <code>IsGlobal</code> = true and references the global choice logical name. In this script the field purpose is: Reusable lifecycle stage for contact communication.	Creates a choice column that reuses an existing global choice. It provides a governed place for reusable lifecycle stage for contact communication.	The global choice logical name must already exist and match exactly.
41	<code>GlobalChoice("aso_journeyparticipationstatu s", "Journey Participation Status", "aso_journeyparticipationstatus", "Uses global Journey Participation Status choice.");</code>	Defines the Journey Participation Status column (<code>aso_journeyparticipationstatus</code>).	Sets <code>IsGlobal</code> = true and references the global choice logical name. In this script the field purpose is: Reusable Customer Insights journey participation status.	Creates a choice column that reuses an existing global choice. It provides a governed place for reusable customer insights journey participation status.	The global choice logical name must already exist and match exactly.
43	<code>Text("aso_customerinsightsjourneyid", "Customer Insights Journey ID", 100, "Latest journey reference.");</code>	Defines the Customer Insights Journey ID column (<code>aso_customerinsightsjourneyid</code>).	Uses <code>StringAttributeMetadata</code> with a maximum length. This is suitable for IDs, names, labels, and references. In this script the field purpose is: Latest related Customer Insights journey identifier.	Creates a short text column. It provides a governed place for latest related customer insights journey identifier.	Wrong length or schema names can create hard-to-change metadata.
44	<code>Text("aso_customerinsightsjourneyname", "Customer Insights Journey Name", 200, "Latest journey name.");</code>	Defines the Customer Insights Journey Name column (<code>aso_customerinsightsjourneyname</code>).	Uses <code>StringAttributeMetadata</code> with a maximum length. This is suitable for IDs, names, labels, and references. In this script the field purpose is: Latest related Customer Insights journey name.	Creates a short text column. It provides a governed place for latest related customer insights journey name.	Wrong length or schema names can create hard-to-change metadata.
45	<code>DateTimeField("aso_customerinsightslasten tryon", "Customer Insights Last Entry On", "Last entry.");</code>	Defines the Customer Insights Last Entry On column (<code>aso_customerinsightslastentryon</code>).	Uses <code>DateTimeAttributeMetadata</code> with <code>DateTimeFormat.DateAndTime</code> and <code>DateTimeBehavior.UserLocal</code> . In this script the field purpose is: Timestamp for last journey entry.	Creates a date/time column. It provides a governed place for timestamp for last journey entry.	Wrong datetime behavior can confuse users in different time zones.
46	<code>DateTimeField("aso_customerinsightslastint eractionon", "Customer Insights Last Interaction On", "Last interaction.");</code>	Defines the Customer Insights Last Interaction On column (<code>aso_customerinsightslastinteractionon</code>).	Uses <code>DateTimeAttributeMetadata</code> with <code>DateTimeFormat.DateAndTime</code> and <code>DateTimeBehavior.UserLocal</code> . In this script the field purpose is: Timestamp for latest Customer Insights interaction.	Creates a date/time column. It provides a governed place for timestamp for latest customer insights interaction.	Wrong datetime behavior can confuse users in different time zones.
48	<code>LocalChoice("aso_customerinsightslastinter actiontype", "Customer Insights Last Interaction Type",</code>	Defines the Customer Insights Last Interaction Type column (<code>aso_customerinsightslastinteractiontype</code>).	Builds a non-global <code>OptionSetMetadata</code> and attaches local <code>OptionMetadata</code> values to the column. In this script the field purpose is: Type of the latest Customer Insights interaction.	Creates a table-specific choice column. It provides a governed place for type of the latest customer insights interaction.	Do not use local choices for values that must be reused across many tables.
49	<code>new[] { "EmailSent", "Open", "Click", "FormSubmit", "Reply", "Unsubscribe", "Bounce", "CustomAction" },</code>	Defines the option values for the preceding local choice column.	The string array is passed into <code>LocalChoice</code> , which converts each value into <code>OptionMetadata</code> .	These labels become the selectable choices in Dataverse.	Changing option text later may affect integrations, filters, and documentation.
50	<code>"Customer Insights last interaction type.");</code>	Provides the description for the Customer Insights Last Interaction Type choice.	This closes the <code>LocalChoice</code> call after the value array.	The description helps makers understand the field purpose in Power Apps.	Descriptions should remain business-readable and not contradict the data model.
52	<code>GlobalChoice("aso_emailconsentstatus", "Email Consent Status", "aso_emailconsentstatus", "Uses global Email Consent Status choice.");</code>	Defines the Email Consent Status column (<code>aso_emailconsentstatus</code>).	Sets <code>IsGlobal</code> = true and references the global choice logical name. In this script the field purpose is: Reusable consent status for communication eligibility.	Creates a choice column that reuses an existing global choice. It provides a governed place for reusable consent status for communication eligibility.	The global choice logical name must already exist and match exactly.
54	<code>Text("aso_complianceprofilename", "Compliance Profile Name", 200, "Profile used.");</code>	Defines the Compliance Profile Name column (<code>aso_complianceprofilename</code>).	Uses <code>StringAttributeMetadata</code> with a maximum length. This is suitable for IDs, names, labels, and references. In this script the field purpose is: Compliance profile used for Customer Insights communication governance.	Creates a short text column. It provides a governed place for compliance profile used for customer insights communication governance.	Wrong length or schema names can create hard-to-change metadata.
55	<code>Text("aso_preferredemailaddress", "Preferred Email Address", 200, "Preferred email address.");</code>	Defines the Preferred Email Address column (<code>aso_preferredemailaddress</code>).	Uses <code>StringAttributeMetadata</code> with a maximum length. This is suitable for IDs, names, labels, and references. In this script the field purpose is: Preferred email address to support communication preference handling.	Creates a short text column. It provides a governed place for preferred email address to support communication preference handling.	Wrong length or schema names can create hard-to-change metadata.
56	<code>DateTimeField("aso_preferencecenterlastvis itedon", "Preference Center Last Visited On", "Preference center last visited timestamp.");</code>	Defines the Preference Center Last Visited On column (<code>aso_preferencecenterlastvisitedon</code>).	Uses <code>DateTimeAttributeMetadata</code> with <code>DateTimeFormat.DateAndTime</code> and <code>DateTimeBehavior.UserLocal</code> . In this script the field purpose is: Timestamp when the preference center was last visited.	Creates a date/time column. It provides a governed place for timestamp when the preference center was last visited.	Wrong datetime behavior can confuse users in different time zones.
57	<code>};</code>	Opens or closes a C# block or object	Braces and terminators define scope and	They keep methods, loops, conditionals,	Missing or extra braces/semicolons cause

		initializer.	statement boundaries in C#.	and metadata objects syntactically valid.	compile errors or wrong structure.
59	foreach (var field in fields)	Starts a loop over each planned column.	The foreach loop processes every AttributeMetadata object in the fields list.	It avoids repeated manual create logic for each column.	If the fields list is wrong, the loop will faithfully create the wrong metadata.
60	{	Opens or closes a C# block or object initializer.	Braces and terminators define scope and statement boundaries in C#.	They keep methods, loops, conditionals, and metadata objects syntactically valid.	Missing or extra braces/semicolons cause compile errors or wrong structure.
61	var logicalName = field.SchemaName!.ToLowerInvariant();	Gets the schema name of the current field and converts it to lowercase.	Dataverse logical names are lowercase; ToLowerInvariant helps compare planned fields with existing fields.	This supports duplicate detection and reliable skip behavior.	Removing lowercase normalization can cause inconsistent comparisons.
63	if (existingAttributes.Contains(logicalName))	Checks whether the current column already exists.	The HashSet lookup uses case-insensitive comparison of existing attribute logical names.	This makes the script safe to rerun after a partial or previous execution.	If this is missing, duplicate-column errors are likely.
64	{	Opens or closes a C# block or object initializer.	Braces and terminators define scope and statement boundaries in C#.	They keep methods, loops, conditionals, and metadata objects syntactically valid.	Missing or extra braces/semicolons cause compile errors or wrong structure.
65	Console.ForegroundColor = ConsoleColor.Yellow;	Changes terminal output color to yellow for skipped items.	Yellow is used when a column already exists and is intentionally skipped.	This shows the script is idempotent and safe to rerun.	A skip is not usually an error; it means the field already exists.
66	Console.WriteLine(\$"SKIP existing: {logicalName}");	Prints that the current column was skipped because it already exists.	The message includes the logical name for traceability.	Operators can distinguish safe skip behavior from failure.	Skipped fields should still be validated if data types are uncertain.
67	Console.ResetColor();	Resets terminal text color back to normal.	Console.ResetColor clears the previous ForegroundColor setting.	It keeps terminal output readable after colored status lines.	If missing, later output may stay red/yellow/green.
68	continue;	Moves to the next field in the loop.	continue skips the create request for an existing column.	This prevents the script from trying to recreate an existing column.	Without it, the script would continue into the create block even after detecting duplicates.
69	}	Opens or closes a C# block or object initializer.	Braces and terminators define scope and statement boundaries in C#.	They keep methods, loops, conditionals, and metadata objects syntactically valid.	Missing or extra braces/semicolons cause compile errors or wrong structure.
71	try	Begins a protected block for creating a column.	C# try/catch captures exceptions from Dataverse SDK requests.	One failed field can be reported clearly without hiding the cause.	Do not suppress errors silently; the catch block must print them.
72	{	Opens or closes a C# block or object initializer.	Braces and terminators define scope and statement boundaries in C#.	They keep methods, loops, conditionals, and metadata objects syntactically valid.	Missing or extra braces/semicolons cause compile errors or wrong structure.
73	Console.WriteLine(\$"Creating: {logicalName} ...");	Prints which field is about to be created.	The log includes the logical name of the current field.	It provides an execution trace for customer IT and troubleshooting.	If the script stops, this helps identify the last attempted field.
75	var request = new CreateAttributeRequest	Creates the Dataverse request object for creating a column.	CreateAttributeRequest is the SDK metadata operation for adding an attribute to an existing table.	This is the core create-column mechanism.	Using the wrong request type will not create table columns.
76	{	Opens or closes a C# block or object initializer.	Braces and terminators define scope and statement boundaries in C#.	They keep methods, loops, conditionals, and metadata objects syntactically valid.	Missing or extra braces/semicolons cause compile errors or wrong structure.
77	EntityName = EntityLogicalName,	Sets the table that receives the new column.	EntityName receives account or contact from the constant.	It ensures the current attribute is created on the intended Dataverse table.	Wrong EntityName creates fields on the wrong table or fails.
78	Attribute = field,	Provides the current column metadata to the create request.	The Attribute property receives a StringAttributeMetadata, MemoAttributeMetadata, PicklistAttributeMetadata, or DateTimeAttributeMetadata object.	This tells Dataverse exactly what type of column to create.	If field metadata is malformed, the create request fails.
79	SolutionUniqueName = SolutionUniqueName	Associates the created column with ASO.Core.	The request property instructs Dataverse to add the metadata component to the specified solution.	This is essential for ALM, export, and customer handover.	If omitted or wrong, fields may not appear in the expected solution.
80	};	Opens or closes a C# block or object initializer.	Braces and terminators define scope and statement boundaries in C#.	They keep methods, loops, conditionals, and metadata objects syntactically valid.	Missing or extra braces/semicolons cause compile errors or wrong structure.
82	service.Execute(request);	Sends the create-column request to Dataverse.	ServiceClient.Execute calls the Dataverse organization service with the SDK request.	This is the line that actually creates the missing column.	Requires sufficient privileges and valid metadata definitions.
84	Console.ForegroundColor = ConsoleColor.Green;	Changes terminal output color to green for successful actions.	Green is used after successful creation or publishing.	This gives an easy visual confirmation during execution.	Green output should still be validated in Power Apps.
85	Console.WriteLine(\$"CREATED: {logicalName}");	Prints a success message for the created column.	The message includes the logical name returned from the current field metadata.	It creates a readable deployment log.	Still validate in Power Apps because terminal success is not a substitute for solution review.
86	Console.ResetColor();	Resets terminal text color back to normal.	Console.ResetColor clears the previous ForegroundColor setting.	It keeps terminal output readable after colored status lines.	If missing, later output may stay red/yellow/green.
87	}	Opens or closes a C# block or object initializer.	Braces and terminators define scope and statement boundaries in C#.	They keep methods, loops, conditionals, and metadata objects syntactically valid.	Missing or extra braces/semicolons cause compile errors or wrong structure.
88	catch (Exception ex)	Starts error handling for a failed column creation.	Any exception from Dataverse or the SDK is captured in ex.	The script can show the exact failing field and error message.	Do not ignore caught errors; fix and rerun.
89	{	Opens or closes a C# block or object initializer.	Braces and terminators define scope and statement boundaries in C#.	They keep methods, loops, conditionals, and metadata objects syntactically valid.	Missing or extra braces/semicolons cause compile errors or wrong structure.
90	Console.ForegroundColor = ConsoleColor.Red;	Changes terminal output color to red for errors.	Console.ForegroundColor controls text color in the terminal.	Operators can quickly identify failures.	Always reset color afterward to avoid confusing output.
91	Console.WriteLine(\$"FAILED: {logicalName}");	Prints which field failed.	The field logical name is included for diagnosis.	This helps IT identify the exact metadata issue.	Common causes include wrong global choice name, permissions, or duplicate field with different type.
92	Console.WriteLine(ex.Message);	Prints the technical error message.	ex.Message contains the SDK or Dataverse exception description.	This is needed for troubleshooting.	Do not remove it; otherwise failures become hard to diagnose.
93	Console.ResetColor();	Resets terminal text color back to normal.	Console.ResetColor clears the previous ForegroundColor setting.	It keeps terminal output readable after colored status lines.	If missing, later output may stay red/yellow/green.
94	}	Opens or closes a C# block or object initializer.	Braces and terminators define scope and statement boundaries in C#.	They keep methods, loops, conditionals, and metadata objects syntactically valid.	Missing or extra braces/semicolons cause compile errors or wrong structure.
95	}	Opens or closes a C# block or object initializer.	Braces and terminators define scope and statement boundaries in C#.	They keep methods, loops, conditionals, and metadata objects syntactically valid.	Missing or extra braces/semicolons cause compile errors or wrong structure.

97	Console.WriteLine();	Writes a blank line to the terminal.	This is a formatting line only.	It makes logs easier to read.	No business or Dataverse effect.
98	Console.WriteLine("Publishing Contact table customizations...");	Prints that Contact customizations are being published.	The script is about to execute PublishXmlRequest for the target table.	Publishing makes metadata changes visible and usable in the maker/runtime experience.	If publishing fails, fields may exist but not be fully available.
100	service.Execute(new PublishXmlRequest	Starts the targeted publish request.	PublishXmlRequest publishes selected Dataverse components rather than all customizations.	It activates the table metadata changes after creation.	Publishing all customizations is broader; targeted publish is cleaner for this script.
101	{	Opens or closes a C# block or object initializer.	Braces and terminators define scope and statement boundaries in C#.	They keep methods, loops, conditionals, and metadata objects syntactically valid.	Missing or extra braces/semicolons cause compile errors or wrong structure.
102	ParameterXml = "<importexportxml><entities><entity>contact</entity></entities></importexportxml>"	Specifies that only the contact table should be published.	The XML tells Dataverse which entity metadata to publish.	It avoids unnecessary publishing of unrelated components.	Wrong XML can publish the wrong table or fail the request.
103	});	Defines part of the C# script structure.	This line contributes to the .NET console application that creates Dataverse metadata.	It supports repeatable and auditable table extension work.	Changing it without understanding the SDK behavior can cause build or metadata errors.
105	Console.ForegroundColor = ConsoleColor.Green;	Changes terminal output color to green for successful actions.	Green is used after successful creation or publishing.	This gives an easy visual confirmation during execution.	Green output should still be validated in Power Apps.
106	Console.WriteLine("Done. Validate in ASO.Core → Tables → Contact → Columns.");	Prints the final validation instruction.	The script ends by reminding the operator to check Power Apps.	Human validation remains mandatory after metadata deployment.	Never skip the validation step before backup or next implementation step.
107	Console.ResetColor();	Resets terminal text color back to normal.	Console.ResetColor clears the previous ForegroundColor setting.	It keeps terminal output readable after colored status lines.	If missing, later output may stay red/yellow/green.
109	static HashSet<string> GetExistingAttributeLogicalNames(ServiceClient service, string entityLogicalName)	Defines a helper function to retrieve existing column logical names.	Returns a HashSet<string> for fast, case-insensitive duplicate checks.	This enables safe reruns and partial recovery.	If this helper is wrong, duplicate detection fails.
110	{	Opens or closes a C# block or object initializer.	Braces and terminators define scope and statement boundaries in C#.	They keep methods, loops, conditionals, and metadata objects syntactically valid.	Missing or extra braces/semicolons cause compile errors or wrong structure.
111	var response = (RetrieveEntityResponse)service.Execute(new RetrieveEntityRequest	Creates or executes a request to retrieve table metadata.	RetrieveEntityRequest asks Dataverse for metadata about a table.	The script needs current attributes before creating new ones.	Incorrect logical names or missing privileges can cause this request to fail.
112	{	Opens or closes a C# block or object initializer.	Braces and terminators define scope and statement boundaries in C#.	They keep methods, loops, conditionals, and metadata objects syntactically valid.	Missing or extra braces/semicolons cause compile errors or wrong structure.
113	LogicalName = entityLogicalName,	Tells the metadata retrieval request which table to inspect.	The helper receives account/contact and assigns it to the request LogicalName.	It retrieves attributes for the correct table.	Wrong table logical name gives wrong duplicate-detection results.
114	EntityFilters = EntityFilters.Attributes,	Requests only attribute/column metadata.	EntityFilters.Attributes limits the response to column definitions.	The script only needs the list of existing columns.	Using the wrong filter may return insufficient metadata.
115	RetrieveAsIfPublished = true	Reads metadata as if unpublished changes are already visible.	This includes latest customizations even before full publication.	It improves duplicate detection during iterative development.	Without it, recent unpublished fields might be missed.
116	});	Defines part of the C# script structure.	This line contributes to the .NET console application that creates Dataverse metadata.	It supports repeatable and auditable table extension work.	Changing it without understanding the SDK behavior can cause build or metadata errors.
118	return response.EntityMetadata.Attributes	Starts returning the existing attribute collection.	The function chains LINQ operations over EntityMetadata.Attributes.	It transforms Dataverse metadata into a usable set of logical names.	If EntityMetadata is null, retrieval failed earlier.
119	.Where(a => ! string.IsNullOrEmpty(a.LogicalName))	Filters out attributes without usable logical names.	LINQ Where removes null, empty, or whitespace logical names.	It avoids invalid values in the duplicate-detection set.	Do not compare null logical names.
120	.Select(a => a.LogicalName!)	Selects only the logical name from each attribute.	The null-forgiving operator tells C# the value is expected not to be null after filtering.	The script only needs names for comparison.	Do not use display names for metadata comparisons.
121	.ToHashSet(StringComparer.Ordinal ignoreCase);	Creates a case-insensitive set of existing logical names.	HashSet gives fast Contains checks and ignores case differences.	It supports safe and efficient skip logic.	Case-sensitive checks could miss existing fields.
122	}	Opens or closes a C# block or object initializer.	Braces and terminators define scope and statement boundaries in C#.	They keep methods, loops, conditionals, and metadata objects syntactically valid.	Missing or extra braces/semicolons cause compile errors or wrong structure.
124	static Label Label(string value) => new(value, LanguageCode);	Defines a helper to create Dataverse labels.	It returns a Label object with the configured LanguageCode.	All display names and descriptions need Dataverse Label objects.	Raw strings are not accepted for DisplayName and Description metadata.
126	static AttributeRequiredLevelManagedProperty Optional()	Defines a helper for optional fields.	It returns AttributeRequiredLevel.None wrapped in a managed property object.	The ASO extension fields are optional at schema level for MVP safety.	Making fields required can break record creation or existing processes.
127	{	Opens or closes a C# block or object initializer.	Braces and terminators define scope and statement boundaries in C#.	They keep methods, loops, conditionals, and metadata objects syntactically valid.	Missing or extra braces/semicolons cause compile errors or wrong structure.
128	return new AttributeRequiredLevelManagedProperty(AttributeRequiredLevel.None);	Marks a column as not required.	Dataverse uses AttributeRequiredLevelManagedProperty to define requirement behavior.	MVP fields should not block users or integrations when values are not yet available.	Do not make AI/SAP/CI fields required unless business process is mature.
129	}	Opens or closes a C# block or object initializer.	Braces and terminators define scope and statement boundaries in C#.	They keep methods, loops, conditionals, and metadata objects syntactically valid.	Missing or extra braces/semicolons cause compile errors or wrong structure.
131	static StringAttributeMetadata Text(string schemaName, string displayName, int maxLength, string description)	Defines the helper that creates text-column metadata.	The helper returns a StringAttributeMetadata object for Dataverse short text fields.	It avoids repeating the same metadata boilerplate for every text field.	MaxLength must be appropriate for the expected value.
132	{	Opens or closes a C# block or object initializer.	Braces and terminators define scope and statement boundaries in C#.	They keep methods, loops, conditionals, and metadata objects syntactically valid.	Missing or extra braces/semicolons cause compile errors or wrong structure.
133	return new StringAttributeMetadata	Returns a new short text metadata object.	This object describes a Dataverse text column to the SDK.	The create request uses this object to create a text field.	If properties are missing, the request may fail or create incomplete metadata.
134	{	Opens or closes a C# block or object initializer.	Braces and terminators define scope and statement boundaries in C#.	They keep methods, loops, conditionals, and metadata objects syntactically valid.	Missing or extra braces/semicolons cause compile errors or wrong structure.
135	SchemaName = schemaName,	Sets the schema name of the column.	Dataverse uses schema/logical names as permanent technical identifiers.	Correct names are essential for ALM, integrations, Power Automate, and forms.	Schema names are difficult or impossible to rename later.
136	DisplayName = Label(displayName),	Sets the human-readable column name.	The helper converts the string into a	This is what makers and users see in the UI.	Display names can be changed later, but

			Dataverse Label object.		schema names should remain stable.
137	Description = Label(description),	Sets the field description.	Descriptions are stored as Dataverse labels and help makers understand the field purpose.	Good descriptions reduce confusion for customer IT and future maintainers.	Vague descriptions reduce handover quality.
138	RequiredLevel = Optional(),	Marks the field as optional.	Calls the Optional helper to return an AttributeRequiredLevelManagedProperty.	Optional fields do not block existing sales/customer records.	Required fields can break existing forms, imports, and integrations.
139	MaxLength = maxLength	Sets the maximum length for short text fields.	The value comes from the helper argument, such as 100 or 200.	It controls how much text Dataverse stores.	Too small a value truncates data; too large can be unnecessary.
140	};	Opens or closes a C# block or object initializer.	Braces and terminators define scope and statement boundaries in C#.	They keep methods, loops, conditionals, and metadata objects syntactically valid.	Missing or extra braces/semicolons cause compile errors or wrong structure.
141	}	Opens or closes a C# block or object initializer.	Braces and terminators define scope and statement boundaries in C#.	They keep methods, loops, conditionals, and metadata objects syntactically valid.	Missing or extra braces/semicolons cause compile errors or wrong structure.
143	static DateTimeAttributeMetadata DateTimeField(string schemaName, string displayName, string description)	Defines the helper that creates date/time-column metadata.	The helper returns DateTimeAttributeMetadata with user-local behavior.	It standardizes timestamps across ASO fields.	Check timezone behavior for integration scenarios.
144	{	Opens or closes a C# block or object initializer.	Braces and terminators define scope and statement boundaries in C#.	They keep methods, loops, conditionals, and metadata objects syntactically valid.	Missing or extra braces/semicolons cause compile errors or wrong structure.
145	return new DateTimeAttributeMetadata	Returns a new date/time metadata object.	This object describes a Dataverse date and time column.	It supports storing Customer Insights and AI run timestamps.	Wrong behavior setting can affect time display.
146	{	Opens or closes a C# block or object initializer.	Braces and terminators define scope and statement boundaries in C#.	They keep methods, loops, conditionals, and metadata objects syntactically valid.	Missing or extra braces/semicolons cause compile errors or wrong structure.
147	SchemaName = schemaName,	Sets the schema name of the column.	Dataverse uses schema/logical names as permanent technical identifiers.	Correct names are essential for ALM, integrations, Power Automate, and forms.	Schema names are difficult or impossible to rename later.
148	DisplayName = Label(displayName),	Sets the human-readable column name.	The helper converts the string into a Dataverse Label object.	This is what makers and users see in the UI.	Display names can be changed later, but schema names should remain stable.
149	Description = Label(description),	Sets the field description.	Descriptions are stored as Dataverse labels and help makers understand the field purpose.	Good descriptions reduce confusion for customer IT and future maintainers.	Vague descriptions reduce handover quality.
150	RequiredLevel = Optional(),	Marks the field as optional.	Calls the Optional helper to return an AttributeRequiredLevelManagedProperty.	Optional fields do not block existing sales/customer records.	Required fields can break existing forms, imports, and integrations.
151	Format = DateTimeFormat.DateAndTime,	Configures the field to store date and time, not date-only.	DateTimeFormat.DateAndTime includes both date and clock time.	Run timestamps and interaction timestamps require time precision.	Date-only would lose important event timing.
152	DateTimeBehavior = DateTimeBehavior.UserLocal	Displays the time according to the user local timezone behavior.	UserLocal is common for user-facing activity and interaction timestamps.	Business users see times in their expected local context.	Integration designs should confirm whether UserLocal or TimeZoneIndependent is better.
153	};	Opens or closes a C# block or object initializer.	Braces and terminators define scope and statement boundaries in C#.	They keep methods, loops, conditionals, and metadata objects syntactically valid.	Missing or extra braces/semicolons cause compile errors or wrong structure.
154	}	Opens or closes a C# block or object initializer.	Braces and terminators define scope and statement boundaries in C#.	They keep methods, loops, conditionals, and metadata objects syntactically valid.	Missing or extra braces/semicolons cause compile errors or wrong structure.
156	static PicklistAttributeMetadata LocalChoice(string schemaName, string displayName, string[] values, string description)	Defines the helper for local choice columns.	It creates a PicklistAttributeMetadata object with an embedded non-global OptionSetMetadata.	It keeps table-specific values local when they are not reusable global choices.	Local choices are harder to reuse across tables.
157	{	Opens or closes a C# block or object initializer.	Braces and terminators define scope and statement boundaries in C#.	They keep methods, loops, conditionals, and metadata objects syntactically valid.	Missing or extra braces/semicolons cause compile errors or wrong structure.
158	var optionSet = new OptionSetMetadata	Creates a new option set object for a local choice.	This object stores the choice metadata before it is attached to the picklist field.	Local choices need an embedded option set definition.	This is different from referencing a global choice.
159	{	Opens or closes a C# block or object initializer.	Braces and terminators define scope and statement boundaries in C#.	They keep methods, loops, conditionals, and metadata objects syntactically valid.	Missing or extra braces/semicolons cause compile errors or wrong structure.
160	IsGlobal = false,	Marks the option set as local to the column.	The choices are stored with the field, not as a reusable global component.	It is appropriate for values specific to one column.	Do not use local choices when many tables need the same values.
161	OptionSetType = OptionSetType.Picklist	Defines the choice type as a single-select picklist.	Picklist means one value can be selected for the column.	The ASO status and interaction fields expect one current value.	Use multi-select only when the business model requires multiple values.
162	};	Opens or closes a C# block or object initializer.	Braces and terminators define scope and statement boundaries in C#.	They keep methods, loops, conditionals, and metadata objects syntactically valid.	Missing or extra braces/semicolons cause compile errors or wrong structure.
164	foreach (var value in values)	Loops over every local choice label.	Each string from the array is converted into an OptionMetadata object.	It avoids writing repeated option-add code manually.	The array values become metadata labels; review spelling before running.
165	{	Opens or closes a C# block or object initializer.	Braces and terminators define scope and statement boundaries in C#.	They keep methods, loops, conditionals, and metadata objects syntactically valid.	Missing or extra braces/semicolons cause compile errors or wrong structure.
166	optionSet.Options.Add(new OptionMetadata(Label(value), null));	Adds one option value to the local choice.	OptionMetadata contains the label and lets Dataverse assign the numeric value automatically.	It creates the actual selectable values.	Do not manually guess option values unless you have an ALM reason.
167	}	Opens or closes a C# block or object initializer.	Braces and terminators define scope and statement boundaries in C#.	They keep methods, loops, conditionals, and metadata objects syntactically valid.	Missing or extra braces/semicolons cause compile errors or wrong structure.
169	return new PicklistAttributeMetadata	Returns a new choice metadata object.	The object describes either a local choice or global-choice-backed column.	It enables governed option values rather than free text.	Choice configuration must match the data model.
170	{	Opens or closes a C# block or object initializer.	Braces and terminators define scope and statement boundaries in C#.	They keep methods, loops, conditionals, and metadata objects syntactically valid.	Missing or extra braces/semicolons cause compile errors or wrong structure.
171	SchemaName = schemaName,	Sets the schema name of the column.	Dataverse uses schema/logical names as permanent technical identifiers.	Correct names are essential for ALM, integrations, Power Automate, and forms.	Schema names are difficult or impossible to rename later.
172	DisplayName = Label(displayName),	Sets the human-readable column name.	The helper converts the string into a Dataverse Label object.	This is what makers and users see in the UI.	Display names can be changed later, but schema names should remain stable.
173	Description = Label(description),	Sets the field description.	Descriptions are stored as Dataverse labels and help makers understand the field	Good descriptions reduce confusion for customer IT and future maintainers.	Vague descriptions reduce handover quality.

			purpose.		
174	RequiredLevel = Optional(),	Marks the field as optional.	Calls the Optional helper to return an AttributeRequiredLevelManagedProperty.	Optional fields do not block existing sales/customer records.	Required fields can break existing forms, imports, and integrations.
175	OptionSet = optionSet	Attaches the local option set to the choice column.	The PicklistAttributeMetadata receives the option set created earlier.	Without this, the choice field would have no values.	Missing option sets cause invalid or empty choice definitions.
176	};	Opens or closes a C# block or object initializer.	Braces and terminators define scope and statement boundaries in C#.	They keep methods, loops, conditionals, and metadata objects syntactically valid.	Missing or extra braces/semicolons cause compile errors or wrong structure.
177	}	Opens or closes a C# block or object initializer.	Braces and terminators define scope and statement boundaries in C#.	They keep methods, loops, conditionals, and metadata objects syntactically valid.	Missing or extra braces/semicolons cause compile errors or wrong structure.
179	static PicklistAttributeMetadata GlobalChoice(string schemaName, string displayName, string globalChoiceName, string description)	Defines the helper for global-choice-backed columns.	It creates PicklistAttributeMetadata that references an existing global option set by name.	It enforces consistent taxonomy across Account, Contact, Lead, and Opportunity.	The global choice must exist before running the script.
180	{	Opens or closes a C# block or object initializer.	Braces and terminators define scope and statement boundaries in C#.	They keep methods, loops, conditionals, and metadata objects syntactically valid.	Missing or extra braces/semicolons cause compile errors or wrong structure.
181	return new PicklistAttributeMetadata	Returns a new choice metadata object.	The object describes either a local choice or global-choice-backed column.	It enables governed option values rather than free text.	Choice configuration must match the data model.
182	{	Opens or closes a C# block or object initializer.	Braces and terminators define scope and statement boundaries in C#.	They keep methods, loops, conditionals, and metadata objects syntactically valid.	Missing or extra braces/semicolons cause compile errors or wrong structure.
183	SchemaName = schemaName,	Sets the schema name of the column.	Dataverse uses schema/logical names as permanent technical identifiers.	Correct names are essential for ALM, integrations, Power Automate, and forms.	Schema names are difficult or impossible to rename later.
184	DisplayName = Label(displayName),	Sets the human-readable column name.	The helper converts the string into a Dataverse Label object.	This is what makers and users see in the UI.	Display names can be changed later, but schema names should remain stable.
185	Description = Label(description),	Sets the field description.	Descriptions are stored as Dataverse labels and help makers understand the field purpose.	Good descriptions reduce confusion for customer IT and future maintainers.	Vague descriptions reduce handover quality.
186	RequiredLevel = Optional(),	Marks the field as optional.	Calls the Optional helper to return an AttributeRequiredLevelManagedProperty.	Optional fields do not block existing sales/customer records.	Required fields can break existing forms, imports, and integrations.
187	OptionSet = new OptionSetMetadata	Defines part of the C# script structure.	This line contributes to the .NET console application that creates Dataverse metadata.	It supports repeatable and auditable table extension work.	Changing it without understanding the SDK behavior can cause build or metadata errors.
188	{	Opens or closes a C# block or object initializer.	Braces and terminators define scope and statement boundaries in C#.	They keep methods, loops, conditionals, and metadata objects syntactically valid.	Missing or extra braces/semicolons cause compile errors or wrong structure.
189	IsGlobal = true,	Marks the option set reference as global.	The field points to an existing reusable global choice component.	This keeps communication and consent states consistent across tables.	The referenced global choice must already exist.
190	Name = globalChoiceName,	Sets the logical name of the existing global choice to reuse.	Dataverse resolves this name to a global option set component.	This ties the column to the governed shared taxonomy.	A typo here causes a global choice not found error.
191	OptionSetType = OptionSetType.Picklist	Defines the choice type as a single-select picklist.	Picklist means one value can be selected for the column.	The ASO status and interaction fields expect one current value.	Use multi-select only when the business model requires multiple values.
192	}	Opens or closes a C# block or object initializer.	Braces and terminators define scope and statement boundaries in C#.	They keep methods, loops, conditionals, and metadata objects syntactically valid.	Missing or extra braces/semicolons cause compile errors or wrong structure.
193	};	Opens or closes a C# block or object initializer.	Braces and terminators define scope and statement boundaries in C#.	They keep methods, loops, conditionals, and metadata objects syntactically valid.	Missing or extra braces/semicolons cause compile errors or wrong structure.
194	}	Opens or closes a C# block or object initializer.	Braces and terminators define scope and statement boundaries in C#.	They keep methods, loops, conditionals, and metadata objects syntactically valid.	Missing or extra braces/semicolons cause compile errors or wrong structure.

Appendix A - Full Account script

```
using Microsoft.Crm.Sdk.Messages;
using Microsoft.PowerPlatform.Dataverse.Client;
using Microsoft.Xrm.Sdk;
using Microsoft.Xrm.Sdk.Messages;
using Microsoft.Xrm.Sdk.Metadata;
```

```
const string DataverseUrl = "https://phoenicarix-ci.crm4.dynamics.com";
const string SolutionUniqueName = "ASOCore";
const string EntityLogicalName = "account";
const int LanguageCode = 1033;
```

```
var connectionString =
    $"AuthType=OAuth;
    Url={DataverseUrl};
    LoginPrompt=Auto;
    ClientId=51f81489-12ee-4a9e-aaae-a2591f45987d;
    RedirectUri=http://localhost";
```

```

using var service = new ServiceClient(connectionString);

if (!service.IsReady)
{
    Console.ForegroundColor = ConsoleColor.Red;
    Console.WriteLine("Connection failed:");
    Console.WriteLine(service.LastError);
    Console.ResetColor();
    return;
}

Console.WriteLine($"Connected to: {DataVERSEUrl}");
Console.WriteLine($"Target solution: {SolutionUniqueName}");
Console.WriteLine($"Target table: {EntityLogicalName}");
Console.WriteLine();

var existingAttributes = GetExistingAttributeLogicalNames(service, EntityLogicalName);

var fields = new List<AttributeMetadata>
{
    Text("aso_sapbusinesspartnerid", "SAP Business Partner ID", 100, "SAP Business Partner reference."),
    Text("aso_sapcustomerid", "SAP Customer ID", 100, "SAP Customer reference."),

    Text("aso_aiaccounthealth", "AI Account Health", 100, "AI account health. Kept as text in MVP because taxonomy may evolve."),
    Memo("aso_aigrowthsummary", "AI Growth Summary", "AI growth summary."),
    Text("aso_airenewalrisk", "AI Renewal Risk", 100, "AI renewal risk. Kept as text in MVP because taxonomy may evolve."),
    DateTimeField("aso_ailastrunon", "AI Last Run On", "Last AI run timestamp."),
    Memo("aso_aisapaccountrichcontext", "AI SAP Account Rich Context", "SAP account rich context."),
    Text("aso_aicorrelationid", "AI Correlation ID", 100, "Last run correlation."),

    GlobalChoice("aso_communicationstate", "Communication State", "aso_communicationstate", "Uses global Communication State choice."),
    GlobalChoice("aso_lifecyclecommunicationstage", "Lifecycle Communication Stage", "aso_lifecyclecommunicationstage", "Uses global Lifecycle Communication Stage choice."),
    GlobalChoice("aso_journeyparticipationstatus", "Journey Participation Status", "aso_journeyparticipationstatus", "Uses global Journey Participation Status choice."),

    Text("aso_customerinsightsjourneyid", "Customer Insights Journey ID", 100, "Latest journey reference."),
    Text("aso_customerinsightsjourneyname", "Customer Insights Journey Name", 200, "Latest journey name."),
    DateTimeField("aso_customerinsightslastentryon", "Customer Insights Last Entry On", "Last entry."),
    DateTimeField("aso_customerinsightslastinteractionon", "Customer Insights Last Interaction On", "Last interaction."),

    LocalChoice("aso_customerinsightslastinteractiontype", "Customer Insights Last Interaction Type",
        new[] { "EmailSent", "Open", "Click", "FormSubmit", "Reply", "Unsubscribe", "Bounce", "CustomAction" },
        "Customer Insights last interaction type."),

    GlobalChoice("aso_emailconsentstatus", "Email Consent Status", "aso_emailconsentstatus", "Uses global Email Consent Status choice."),
    Text("aso_complianceprofilename", "Compliance Profile Name", 200, "Profile used."),
    Memo("aso_communicationholdreason", "Communication Hold Reason", "Suppression / hold reason.")
};

foreach (var field in fields)
{
    var logicalName = field.SchemaName!.ToLowerInvariant();

    if (existingAttributes.Contains(logicalName))
    {
        Console.ForegroundColor = ConsoleColor.Yellow;
        Console.WriteLine($"SKIP existing: {logicalName}");
        Console.ResetColor();
        continue;
    }
}

```

```

    }

    try
    {
        Console.WriteLine($"Creating: {logicalName} ...");

        var request = new CreateAttributeRequest
        {
            EntityName = EntityLogicalName,
            Attribute = field,
            SolutionUniqueName = SolutionUniqueName
        };

        service.Execute(request);

        Console.ForegroundColor = ConsoleColor.Green;
        Console.WriteLine($"CREATED: {logicalName}");
        Console.ResetColor();
    }
    catch (Exception ex)
    {
        Console.ForegroundColor = ConsoleColor.Red;
        Console.WriteLine($"FAILED: {logicalName}");
        Console.WriteLine(ex.Message);
        Console.ResetColor();
    }
}

Console.WriteLine();
Console.WriteLine("Publishing Account table customizations...");

service.Execute(new PublishXmlRequest
{
    ParameterXml = "<importexportxml><entities><entity>account</entity></entities></importexportxml>"
});

Console.ForegroundColor = ConsoleColor.Green;
Console.WriteLine("Done. Validate in ASO.Core → Tables → Account → Columns.");
Console.ResetColor();

static HashSet<string> GetExistingAttributeLogicalNames(ServiceClient service, string entityLogicalName)
{
    var response = (RetrieveEntityResponse)service.Execute(new RetrieveEntityRequest
    {
        LogicalName = entityLogicalName,
        EntityFilters = EntityFilters.Attributes,
        RetrieveAsIfPublished = true
    });

    return response.EntityMetadata.Attributes
        .Where(a => !string.IsNullOrEmpty(a.LogicalName))
        .Select(a => a.LogicalName!)
        .ToHashSet(StringComparer.OrdinalIgnoreCase);
}

static Label Label(string value) => new(value, LanguageCode);

static AttributeRequiredLevelManagedProperty Optional()
{
    return new AttributeRequiredLevelManagedProperty(AttributeRequiredLevel.None);
}

```

```

}

static StringAttributeMetadata Text(string schemaName, string displayName, int maxLength, string description)
{
    return new StringAttributeMetadata
    {
        SchemaName = schemaName,
        DisplayName = Label(displayName),
        Description = Label(description),
        RequiredLevel = Optional(),
        MaxLength = maxLength
    };
}

static MemoAttributeMetadata Memo(string schemaName, string displayName, string description)
{
    return new MemoAttributeMetadata
    {
        SchemaName = schemaName,
        DisplayName = Label(displayName),
        Description = Label(description),
        RequiredLevel = Optional(),
        MaxLength = 4000
    };
}

static DateTimeAttributeMetadata DateTimeField(string schemaName, string displayName, string description)
{
    return new DateTimeAttributeMetadata
    {
        SchemaName = schemaName,
        DisplayName = Label(displayName),
        Description = Label(description),
        RequiredLevel = Optional(),
        Format = DateTimeFormat.DateAndTime,
        DateTimeBehavior = DateTimeBehavior.UserLocal
    };
}

static PicklistAttributeMetadata LocalChoice(string schemaName, string displayName, string[] values, string description)
{
    var optionSet = new OptionSetMetadata
    {
        IsGlobal = false,
        OptionSetType = OptionSetType.Picklist
    };

    foreach (var value in values)
    {
        optionSet.Options.Add(new OptionMetadata(Label(value), null));
    }

    return new PicklistAttributeMetadata
    {
        SchemaName = schemaName,
        DisplayName = Label(displayName),
        Description = Label(description),
        RequiredLevel = Optional(),
        OptionSet = optionSet
    };
}

```

```

static PicklistAttributeMetadata GlobalChoice(string schemaName, string displayName, string globalChoiceName, string description)
{
    return new PicklistAttributeMetadata
    {
        SchemaName = schemaName,
        DisplayName = Label(displayName),
        Description = Label(description),
        RequiredLevel = Optional(),
        OptionSet = new OptionSetMetadata
        {
            IsGlobal = true,
            Name = globalChoiceName,
            OptionSetType = OptionSetType.Picklist
        }
    };
}

```

Appendix B - Full Contact script

```

using Microsoft.Crm.Sdk.Messages;
using Microsoft.PowerPlatform.Dataaverse.Client;
using Microsoft.Xrm.Sdk;
using Microsoft.Xrm.Sdk.Messages;
using Microsoft.Xrm.Sdk.Metadata;

const string DataaverseUrl = "https://phoenicarix-ci.crm4.dynamics.com";
const string SolutionUniqueName = "ASOCore";
const string EntityLogicalName = "contact";
const int LanguageCode = 1033;

var connectionString =
    $"@AuthType=OAuth;
    Url={DataaverseUrl};
    LoginPrompt=Auto;
    ClientId=51f81489-12ee-4a9e-aaae-a2591f45987d;
    RedirectUri=http://localhost";

using var service = new ServiceClient(connectionString);

if (!service.IsReady)
{
    Console.ForegroundColor = ConsoleColor.Red;
    Console.WriteLine("Connection failed:");
    Console.WriteLine(service.LastError);
    Console.ResetColor();
    return;
}

Console.WriteLine($"Connected to: {DataaverseUrl}");
Console.WriteLine($"Target solution: {SolutionUniqueName}");
Console.WriteLine($"Target table: {EntityLogicalName}");
Console.WriteLine();

var existingAttributes = GetExistingAttributeLogicalNames(service, EntityLogicalName);

var fields = new List<AttributeMetadata>
{
    GlobalChoice("aso_communicationstate", "Communication State", "aso_communicationstate", "Uses global Communication State choice."),
    GlobalChoice("aso_lifecyclecommunicationstage", "Lifecycle Communication Stage", "aso_lifecyclecommunicationstage", "Uses global Lifecycle Communication Stage choice."),
    GlobalChoice("aso_journeyparticipationstatus", "Journey Participation Status", "aso_journeyparticipationstatus", "Uses global Journey Participation Status choice."),

```

```

Text("aso_customerinsightsjourneyid", "Customer Insights Journey ID", 100, "Latest journey reference."),
Text("aso_customerinsightsjourneyname", "Customer Insights Journey Name", 200, "Latest journey name."),
DateTimeField("aso_customerinsightslastentryon", "Customer Insights Last Entry On", "Last entry."),
DateTimeField("aso_customerinsightslastinteractionon", "Customer Insights Last Interaction On", "Last interaction."),

LocalChoice("aso_customerinsightslastinteractiontype", "Customer Insights Last Interaction Type",
    new[] { "EmailSent", "Open", "Click", "FormSubmit", "Reply", "Unsubscribe", "Bounce", "CustomAction" },
    "Customer Insights last interaction type."),

GlobalChoice("aso_emailconsentstatus", "Email Consent Status", "aso_emailconsentstatus", "Uses global Email Consent Status choice."),

Text("aso_complianceprofilename", "Compliance Profile Name", 200, "Profile used."),
Text("aso_preferredemailaddress", "Preferred Email Address", 200, "Preferred email address."),
DateTimeField("aso_preferencecenterlastvisitedon", "Preference Center Last Visited On", "Preference center last visited timestamp.")
};

foreach (var field in fields)
{
    var logicalName = field.SchemaName!.ToLowerInvariant();

    if (existingAttributes.Contains(logicalName))
    {
        Console.ForegroundColor = ConsoleColor.Yellow;
        Console.WriteLine($"SKIP existing: {logicalName}");
        Console.ResetColor();
        continue;
    }

    try
    {
        Console.WriteLine($"Creating: {logicalName} ...");

        var request = new CreateAttributeRequest
        {
            EntityName = EntityLogicalName,
            Attribute = field,
            SolutionUniqueName = SolutionUniqueName
        };

        service.Execute(request);

        Console.ForegroundColor = ConsoleColor.Green;
        Console.WriteLine($"CREATED: {logicalName}");
        Console.ResetColor();
    }
    catch (Exception ex)
    {
        Console.ForegroundColor = ConsoleColor.Red;
        Console.WriteLine($"FAILED: {logicalName}");
        Console.WriteLine(ex.Message);
        Console.ResetColor();
    }
}

Console.WriteLine();
Console.WriteLine("Publishing Contact table customizations...");

service.Execute(new PublishXmlRequest

```



```

{
    ParameterXml = "<importexportxml><entities><entity>contact</entity></entities></importexportxml>"
});

Console.ForegroundColor = ConsoleColor.Green;
Console.WriteLine("Done. Validate in ASO.Core → Tables → Contact → Columns.");
Console.ResetColor();

static HashSet<string> GetExistingAttributeLogicalNames(ServiceClient service, string entityLogicalName)
{
    var response = (RetrieveEntityResponse)service.Execute(new RetrieveEntityRequest
    {
        LogicalName = entityLogicalName,
        EntityFilters = EntityFilters.Attributes,
        RetrieveAsIfPublished = true
    });

    return response.EntityMetadata.Attributes
        .Where(a => !string.IsNullOrEmpty(a.LogicalName))
        .Select(a => a.LogicalName!)
        .ToHashSet(StringComparer.OrdinalIgnoreCase);
}

static Label Label(string value) => new(value, LanguageCode);

static AttributeRequiredLevelManagedProperty Optional()
{
    return new AttributeRequiredLevelManagedProperty(AttributeRequiredLevel.None);
}

static StringAttributeMetadata Text(string schemaName, string displayName, int maxLength, string description)
{
    return new StringAttributeMetadata
    {
        SchemaName = schemaName,
        DisplayName = Label(displayName),
        Description = Label(description),
        RequiredLevel = Optional(),
        MaxLength = maxLength
    };
}

static DateTimeAttributeMetadata DateTimeField(string schemaName, string displayName, string description)
{
    return new DateTimeAttributeMetadata
    {
        SchemaName = schemaName,
        DisplayName = Label(displayName),
        Description = Label(description),
        RequiredLevel = Optional(),
        Format = DateTimeFormat.DateAndTime,
        DateTimeBehavior = DateTimeBehavior.UserLocal
    };
}

static PicklistAttributeMetadata LocalChoice(string schemaName, string displayName, string[] values, string description)
{
    var optionSet = new OptionSetMetadata
    {
        IsGlobal = false,
        OptionSetType = OptionSetType.Picklist
    };
}

```

```

        foreach (var value in values)
        {
            optionSet.Options.Add(new OptionMetadata(Label(value), null));
        }

        return new PicklistAttributeMetadata
        {
            SchemaName = schemaName,
            DisplayName = Label(displayName),
            Description = Label(description),
            RequiredLevel = Optional(),
            OptionSet = optionSet
        };
    }

    static PicklistAttributeMetadata GlobalChoice(string schemaName, string displayName, string globalChoiceName, string description)
    {
        return new PicklistAttributeMetadata
        {
            SchemaName = schemaName,
            DisplayName = Label(displayName),
            Description = Label(description),
            RequiredLevel = Optional(),
            OptionSet = new OptionSetMetadata
            {
                IsGlobal = true,
                Name = globalChoiceName,
                OptionSetType = OptionSetType.Picklist
            }
        };
    }
}

```